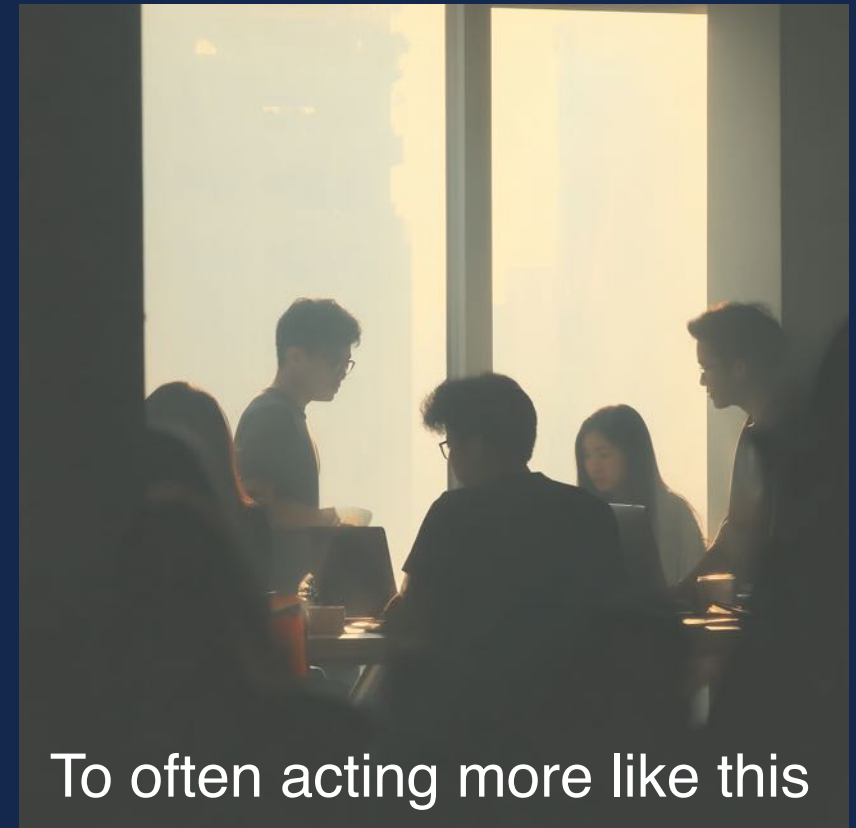
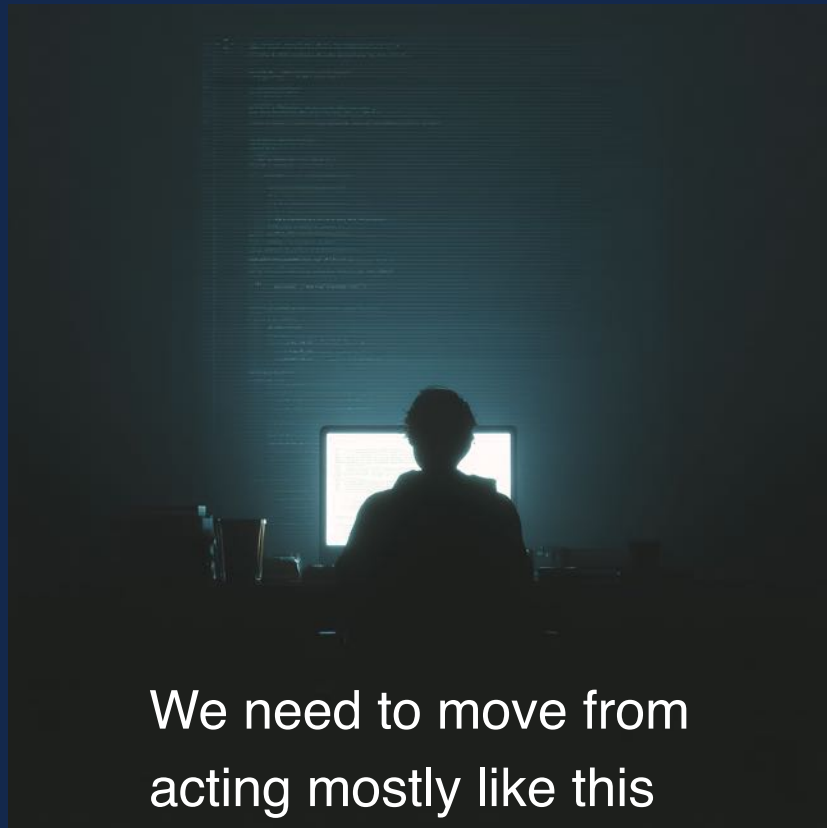


```
//CSE  
110;
```

CSE 110 - Week 2

with Thomas Powell

We Must Transition (Somewhat) Now



Now it's time to ...

ASSEMBLE THE TEAMS!



The Spoiler

“The business we're in is more sociological than technological, more dependent on workers' abilities to communicate with each other than their abilities to communicate with machines.”

Peopleware

Tom DeMarco and Timothy R. Lister



The Power of Stable Teams

Teams are harder to build than projects. Therefore, it is better to form persistent teams that move together from one project to the next and can take on more than one project at a time. The goal in forming a team is to give that team enough time to gel, and then keep it together as an engine for getting many projects done.

The Clean Coder
Robert C. Martin



Teams beat Individuals



Allen Holub @allenholub · Sep 4



The fallacy that hiring "A" people (or "rock stars") will do anything at all useful is predicated on the idea that we all work as isolated individuals. The thinking makes sense only if every product is created by a single person. As soon as you have 2, the system becomes dominant

 10

 17

 140



Team Design is Hard!



Steve @SteveTwips · Sep 6



Every day I'm more convinced that the majority of time taken to produce software is due to assumptions about how we must split-up the work and team.



4



18



104



Project Hinting

Project Time!?! Let's Goooooooooooo!



Project Generally

- **Smallish*** so we can do it well within our allotted time
- **Constrained*** but with room for creativity
 - Minimalism - platform focused, low or no dependencies
- **Production quality***
 - Appropriate SWE (releases, tests, docs, etc.)
 - Long-term thinking - it can revisit you!

* suggests a range of outcomes and expectations

Constraints on Purpose as They Aid Creativity

Design teams know that something needs to be produced to fulfill specific goals and objectives, often within specific constraints and criteria-budget limitations, specified materials, and the amount of time that can be spent on the project. The excitement of design is being creative within these constraints.

Visual Teams

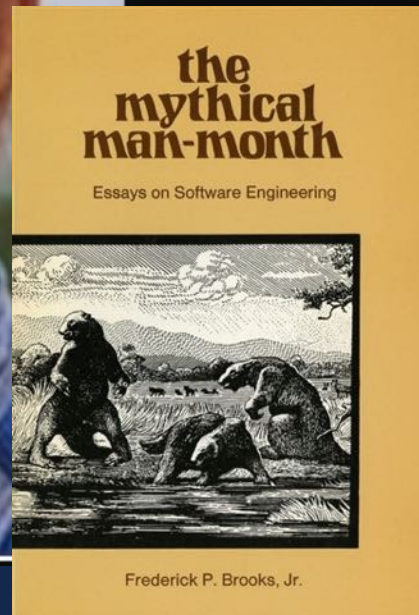
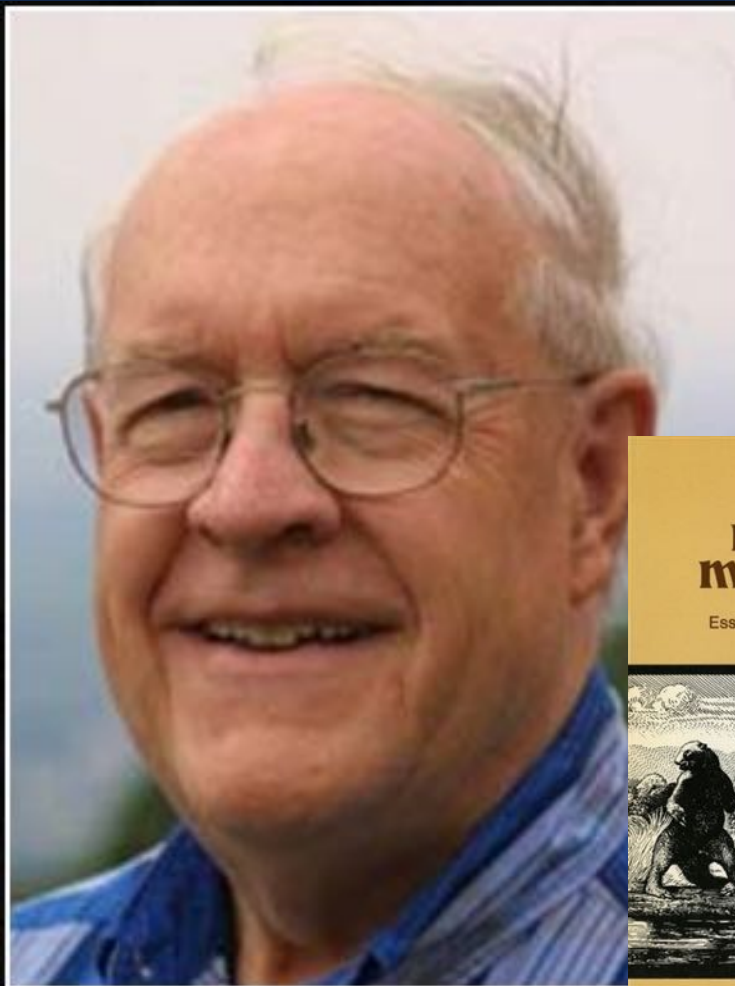
David Sibbet



You must Spec AND then Build

- Unlike a PA you aren't going to be handed a spec and rubric* you will be defining these in reaction to many things presented.
- SWE Original Sin: Assuming someone else did the hard work of doing design right as they write the JIRA ticket or project plan is a **very big assumption most SWEs make today. This mistake is leading to tremendous failures and frustrations.** We must learn not just how to make software but what software to make!

Extreme Danger Approaching



The hardest single part of building a software system is deciding precisely what to build.

— Fred Brooks —

Warm-Up Exercise Ahead

- This term, we will do a single week+ crash course sprint to:
 - Stabilize & bond the team early and often
 - Reveal tech or team concerns to address
 - Address the AI use pros and cons

Warning: A warm-up is just that a warm-up, it is not going to be great it likely will be a bit cringe. Allow it! The only way a team can do poorly on the warm-up is not trying earnestly.

But Stop! We're Not Ready

- Get the team started and get focused
- Rough Plan
 - Meet everyone
 - Define your team rules
 - Define your team vision
 - Start to plan how we get to know each other
 - Do some “drills” - Labs and our LLM and AI exploratory work
 - Have a little fun!
- First up let's consult some furry friends and ask them what they think we should do before getting serious again

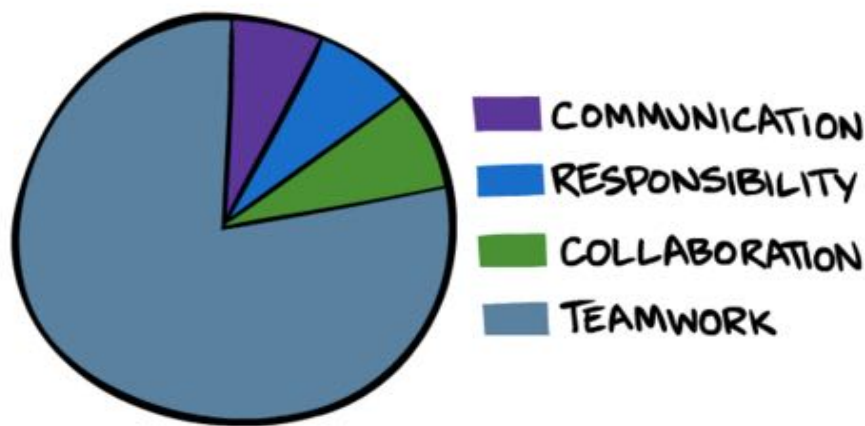
Animal Deck Time!

```
//CSE  
110;
```

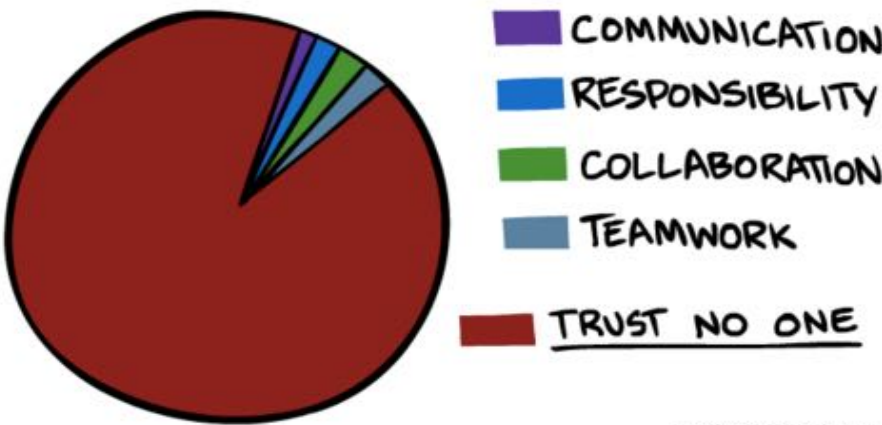
On Groups

with Thomas Powell

WHAT GROUP PROJECTS ARE SUPPOSED TO TEACH YOU

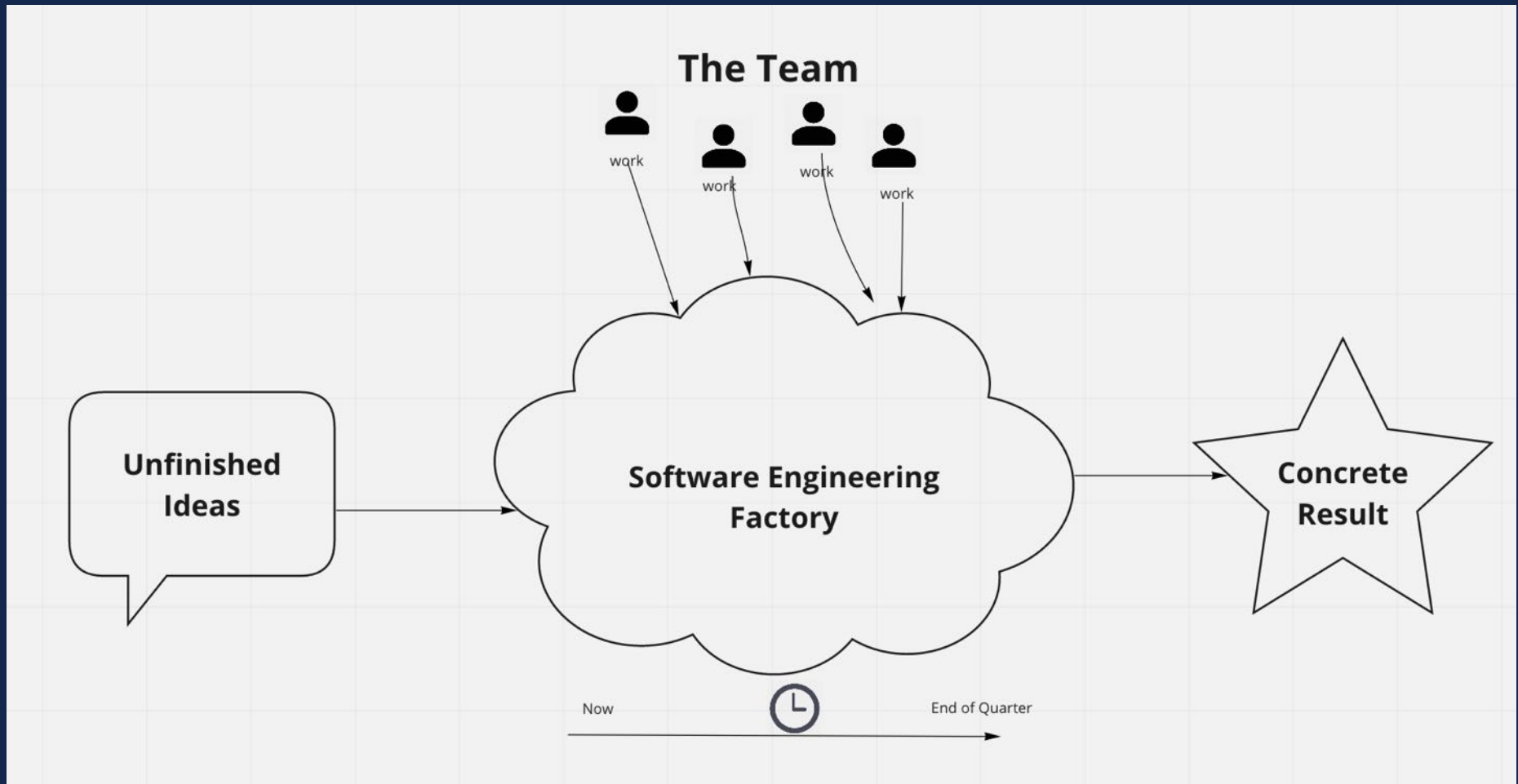


WHAT GROUP PROJECTS TAUGHT ME



endlessorigami.com

The Software Process



Is This Our Process?

Step 1 – Make a team

Step 2 – Somebody Tells Us What to Do

Step 3 – We code like crazy

Step 4 – Release

Step 5 – Profit

- Clearly that is ridiculous but even if was about right step 3 is about more than just coding it requires coordination and communication more than anything
- Communication isn't that easy!

The Sad Reality of Group Projects

- **Reality:** Forced Collaboration may go poorly
- **Positive Point:** Test your metal, learn from mistakes
- **Negative Point:** Unprepared for the experience to likely succeed
- Truthfully, the Professors' have a hand in this too as we rarely give you any significant amount of coaching on how to succeed

Stages of Group Development

Tuckman (1965) defined four stages that a group make go thru in development. A fifth is commonly added

1. Forming
2. Storming
3. Norming
4. Performing
5. Adjourning (the wind-up step)

Note: There really isn't a set number of name of steps and numerous models exist (https://en.wikipedia.org/wiki/Group_development) a key point is just to expect that a group evolves and should be self-aware if it is to become high functioning

https://en.wikipedia.org/wiki/Group_development

https://en.wikipedia.org/wiki/Team_building

Team Communication Part 1

Communication is innate or not...

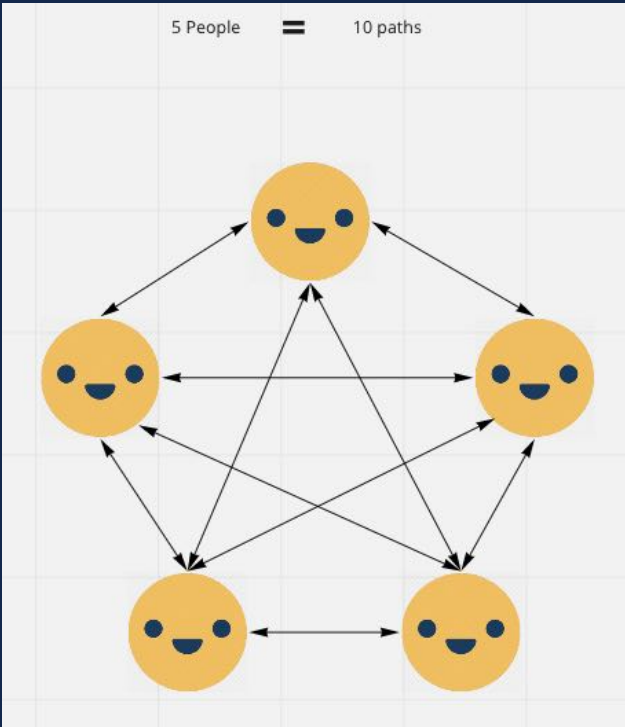
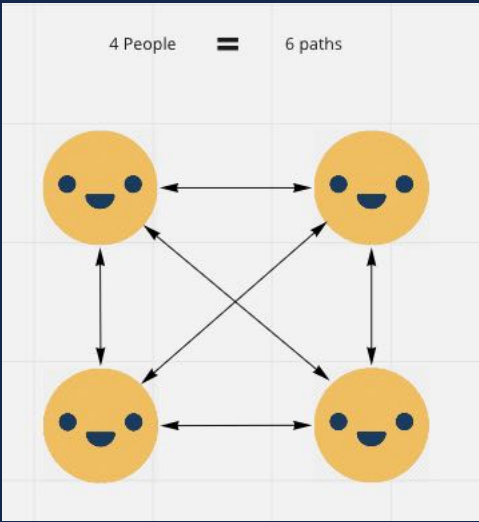
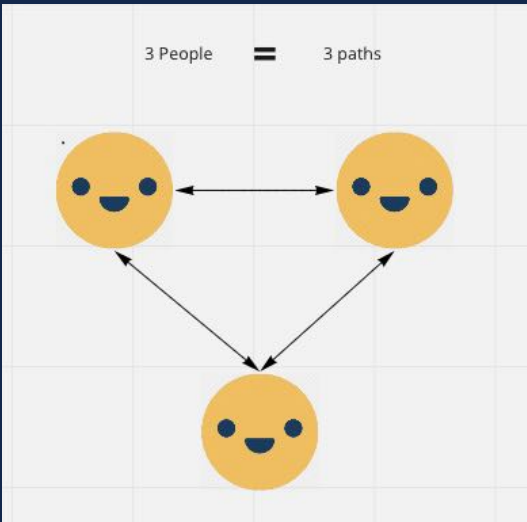
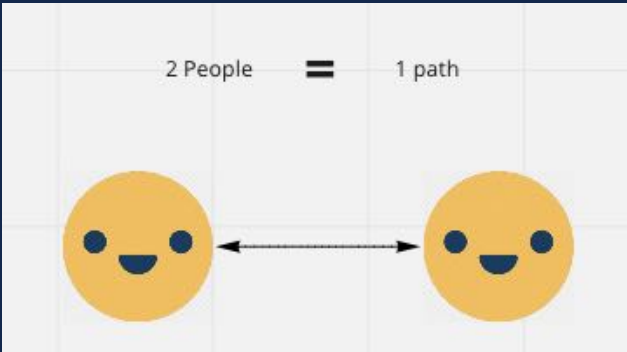
- The number one thing reported from last quarter -
COMMUNICATION
- Now let's reveal can we learn how to talk/chat/email with others or are we doomed to get by or even be good at it
- Studies suggest it is a learned activity. Obviously, there is a question of disposition which is also true. Disposition does not generally indicate inability though there are extremes

Communication is Like Passing

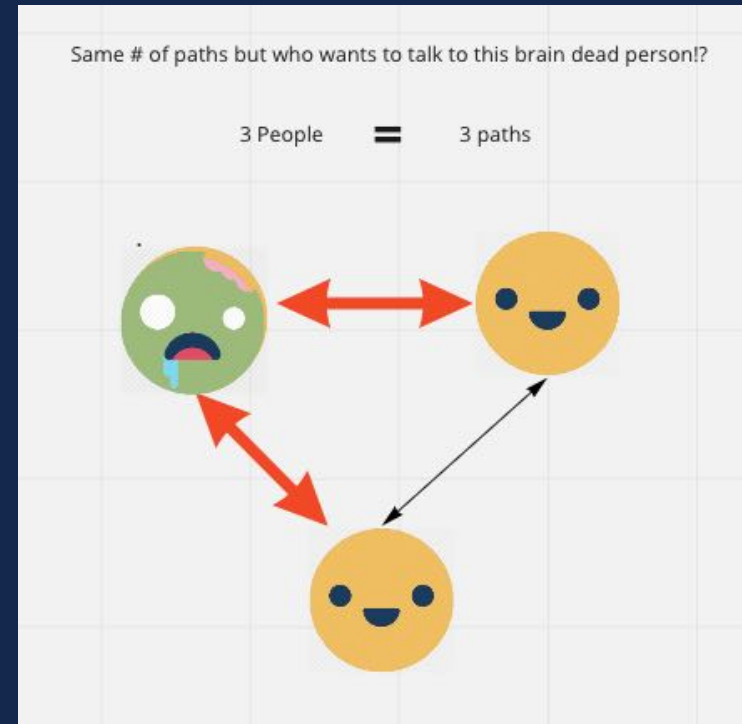
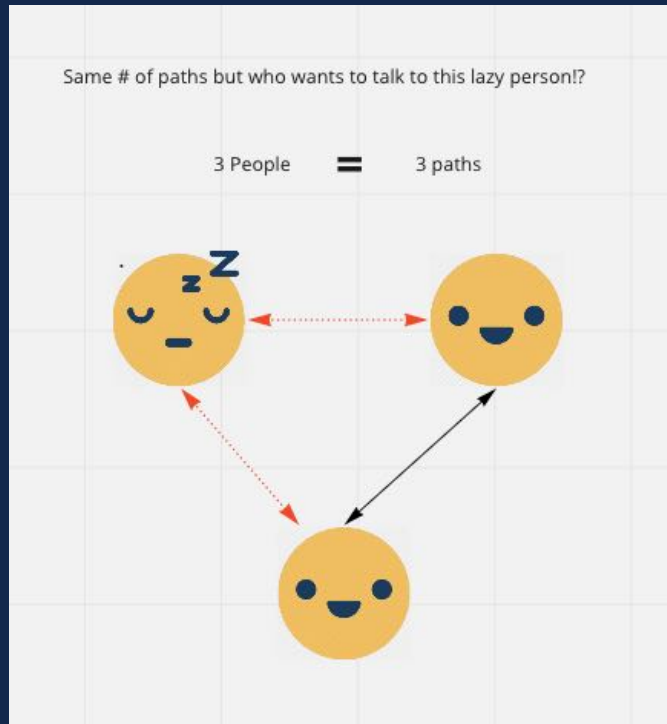
- Since I love soccer lets assume the idea of passing for a software team is communicating or working with each other
- In general, in dev like in Football we can't play alone. Yet when we play, we also can't play "bunch ball"*
- We need to learn how to pass, but this gets harder the more players we add

* Mob Programming will be something we look at briefly

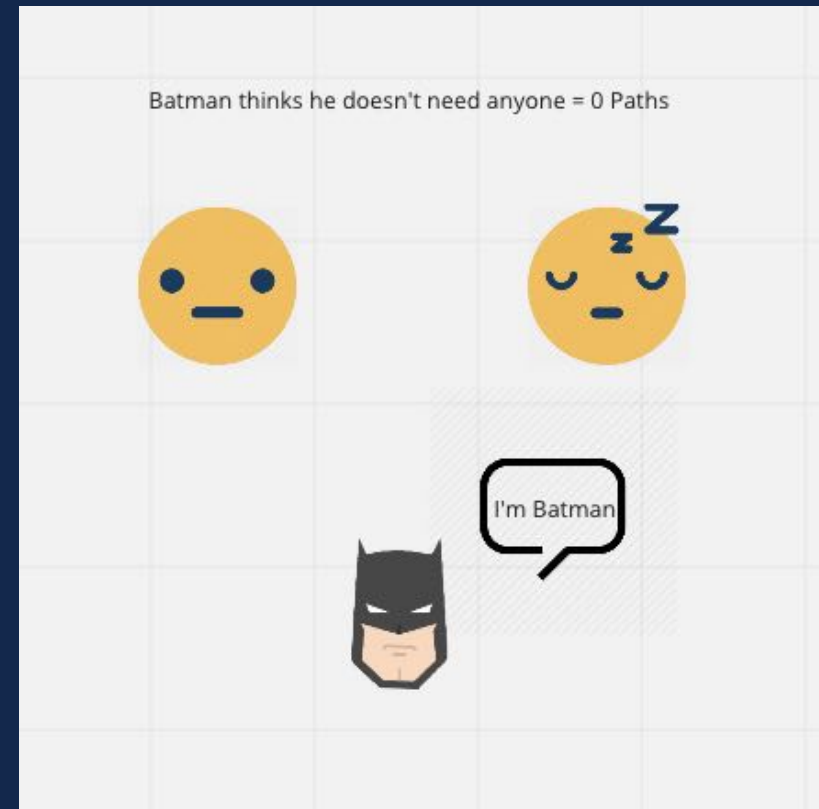
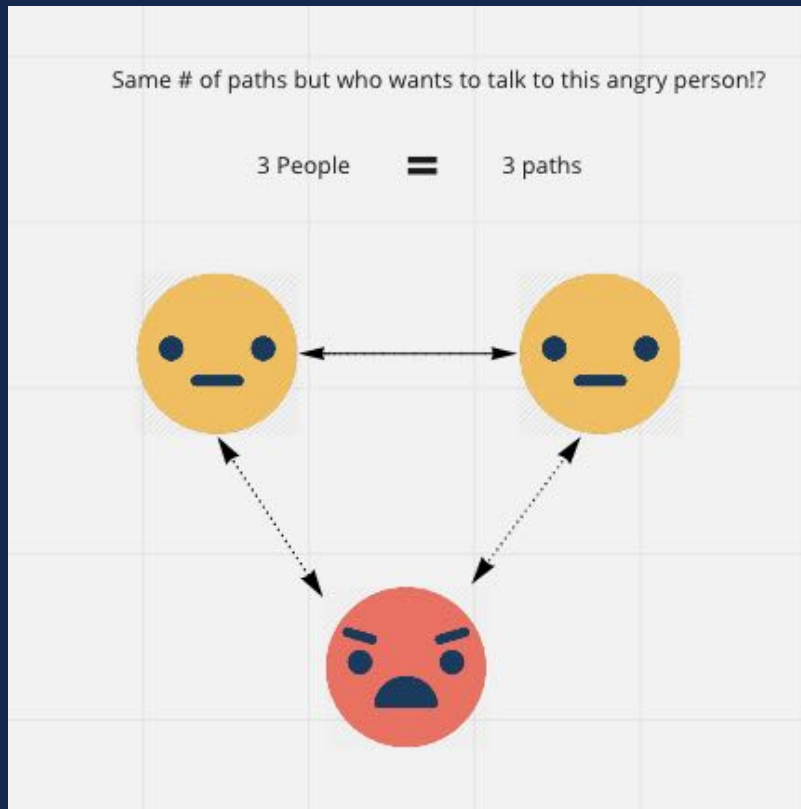
Team Dynamics



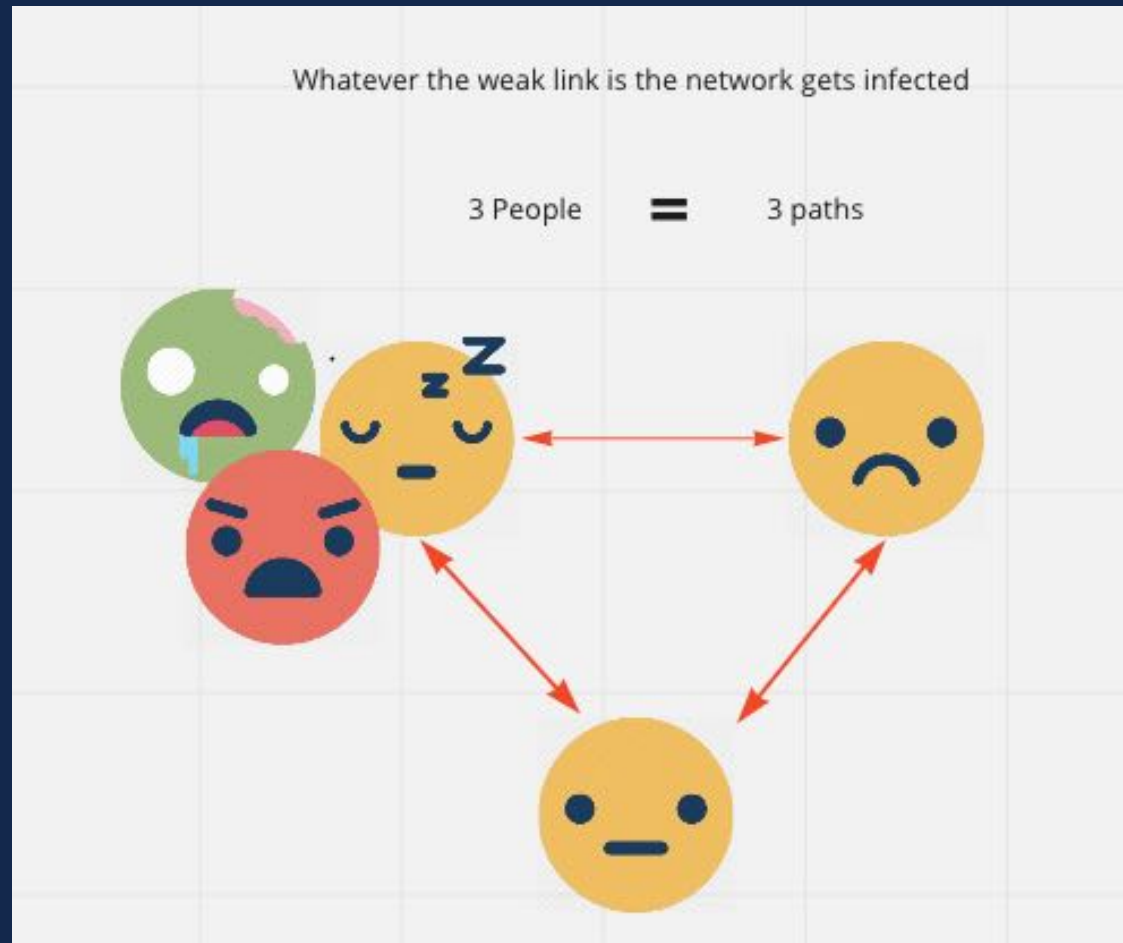
Team Dynamics Contd.



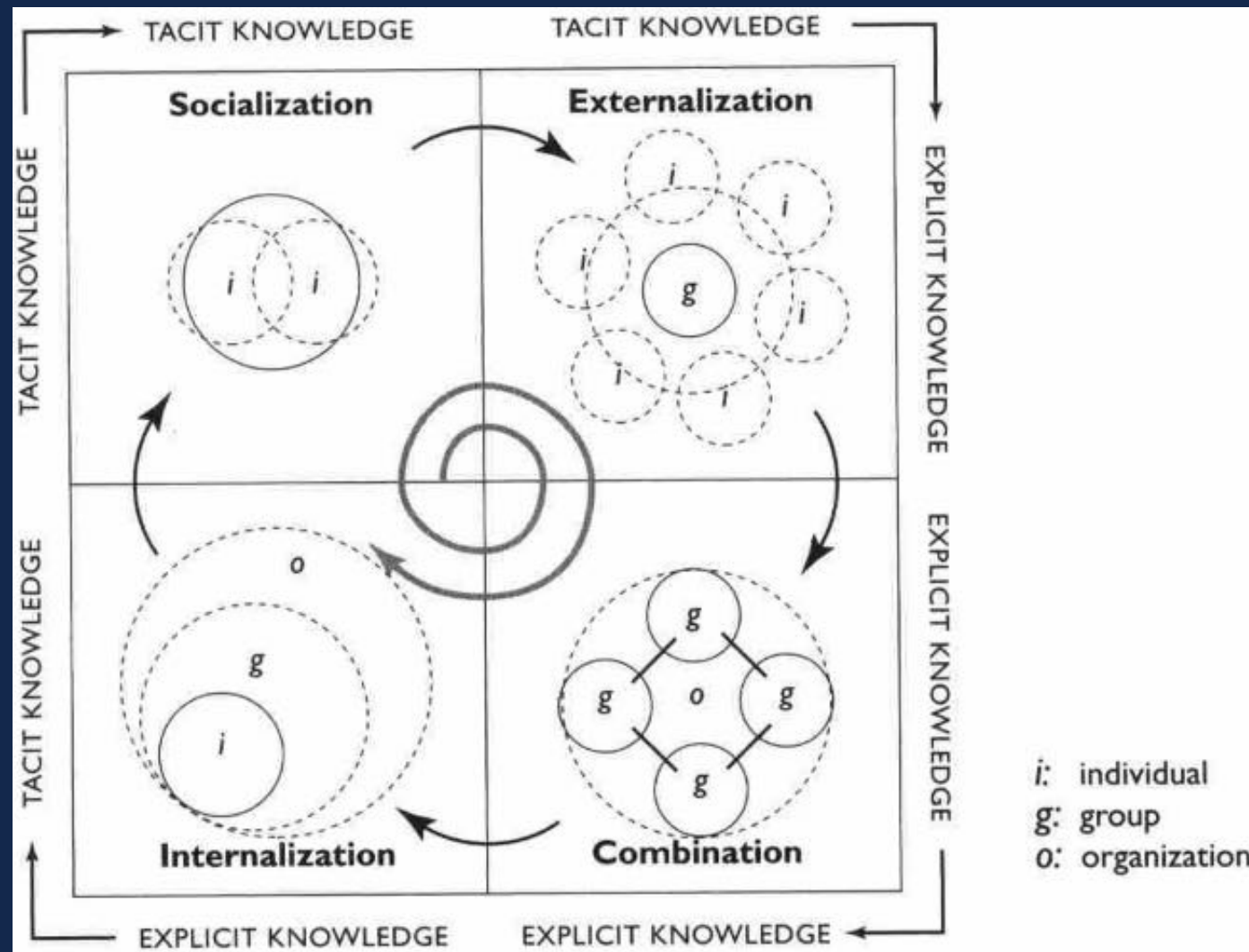
Team Dynamics Contd.



Team Dynamics Contd.



Applied Thought: Alternate and Expanded Knowledge Flows



"Spiral Evolution of Knowledge Conversion and Self-transcending Process" from Nonaka and Konno

Team Size Part 1

Good Question!



Jason Gorman @jasongorman · Oct 11, 2021



I've seen developers write software by themselves > 200 KLOC in a year or two. I've seen teams of less than a dozen write software > 1 MLOC in similar timespans. So when I see teams of literally hundreds of developers, I can't help wondering "WTF are they building?"



Team Size Effects

- As the communication lines increase direct communication between all parties becomes unwieldy and inefficient
- To mitigate size problems we often organize under hierarchies with leaders, managers, higher managers and so on
- We see these patterns in the military, government, crime syndicates, and all sorts of large scale organizations

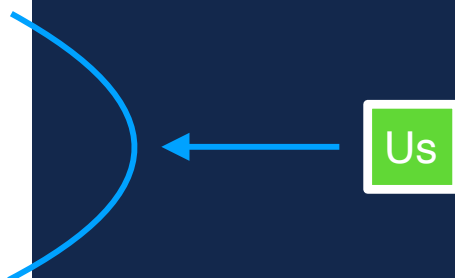
Team Size Effects Contd.

- Personally I tend to see size step functions at 1,2,5,10,30,50,100, 250, 500, 1000, +
- My breakdown is my own, but it isn't far off military and you see other Agile frameworks adopting ideas.
- Example: Crystal Method



Crystal Color Breakdown

1. **Crystal Clear**- The team consists of only 1-6 members that is suitable for short-term projects where members work out in a single workspace.
2. **Crystal Yellow**- It has a small team size of 7-20 members, where feedback is taken from Real Users. This variant involves automated testing which resolves bugs faster and reduces the use of too much documentation.
3. **Crystal Orange**- It has a team size of 21-40 members, where the team is split according to their functional skills. Here the project generally lasts for 1-2 years and the release is required every 3 to 4 months.
4. **Crystal Orange Web**- It has also a team size of 21-40 members were the projects that have a continually evolving code base that is being used by the public. It is also similar to Crystal Orange but here they do not deal with a single project but a series of initiatives that required programming.
5. **Crystal Red**- The software development is led by 40-80 members where the teams can be formed and divided according to requirements.
6. **Crystal Maroon**- It involves large-sized projects where the team size is 80-200 members and where methods are different and as per the requirement of the software.
7. **Crystal Diamond & Sapphire**- This variant is used in large projects where there is a potential risk to human life.



Us

More Crystal Method Details (Agile Preview)

1. **Frequent Delivery-** It allows you regularly deliver the products and test code to real users. Without this, you might build a product that nobody needs.
2. **Reflective Improvement-** No matter how good you have done or how bad you have done. Since there are always areas where the product can be improved, so the teams can implement to improve their future practices.
3. **Osmotic Communication-** Alistair stated that having the teams in the same physical phase is very much important as it allows information to flow in between members of a team as in osmosis.
4. **Personal Safety-** There are no bad suggestions in a crystal team, team members should feel safe to discuss ideas openly without any fear.
5. **Focus-** Each member of the team knows exactly what to do, which enables them to focus their attention. This boosts team interaction and works towards the same goal.
6. **Easy access to expert users-** It enhances team communication with users and gets regular feedback from real users.
7. **Technical tooling-** It contains very specific technical tools which to be used by the software development team during testing, management, and configuration. These tools make it enable the team to identify any error within less time.

Team Composition

Team Org - Specialists or Generalists Aligned to the Situation!

- There are two major trends I see in team organization patterns
 1. The idea that all the members are somewhat interchangeable (Agile - misinterpreted in my view)
 2. The idea that the members are a bit more diverse in approach and even ability in some cases (the SEAL team, soccer team, etc. mentality)

Specialists or Generalists

- Some people believe that an Agile team is a group of devs by themselves all able to do roughly the same things
- In my opinion this is like a soccer team with all midfielders and is a terrible idea that actually turns into what I promote instead informally - a group of like balanced and supportive specialists
 - Gamers in the room? Now let's discuss the success of a team with all "tanks", all "healers", etc.
- I think if you had all the same type you gain some redundancy, but quite likely you build a team that needs considerable outside support to function properly. It might be good as a "work squad" but will not be fully self-sufficient. To me with the whole micro services fixation makes the generic squad a failure and the highly influential Team Topologies book seems to support my view and experience

Example” Belbin Team Roles

- High functioning teams are made up of individuals who are diverse in skill and attitude.
- Numerous people have studied what are typical traits and attitudes of teams and attempted to label them
- Obviously we can debate names, stereotyping, and implications of categorizations of any model, but the Belbin model seems useful and fairly innocuous at least for broad segmentation

<https://www.belbin.com/about/belbin-team-roles>

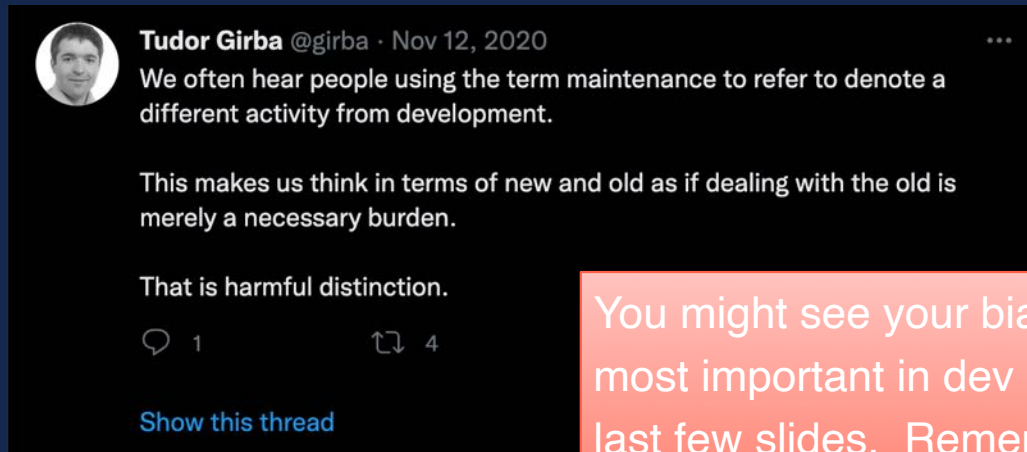
 Resource Investigator Uses their inquisitive nature to find ideas to bring back to the team. Strengths: Outgoing, enthusiastic. Explores opportunities and develops contacts. Allowable weaknesses: Might be over-optimistic, and can lose interest once the initial enthusiasm has passed. Don't be surprised to find that: They might forget to follow up on a lead.	 Teamworker Helps the team to gel, using their versatility to identify the work required and complete it on behalf of the team. Strengths: Co-operative, perceptive and diplomatic. Listens and averts friction. Allowable weaknesses: Can be indecisive in crunch situations and tends to avoid confrontation. Don't be surprised to find that: They might be hesitant to make unpopular decisions.	 Co-ordinator Needed to focus on the team's objectives, draw out team members and delegate work appropriately. Strengths: Mature, confident, identifies talent. Clarifies goals. Allowable weaknesses: Can be seen as manipulative and might offload their own share of the work. Don't be surprised to find that: They might over-delegate, leaving themselves little work to do.
 Plant Tends to be highly creative and good at solving problems in unconventional ways. Strengths: Creative, imaginative, free-thinking, generates ideas and solves difficult problems. Allowable weaknesses: Might ignore incidentals, and may be too preoccupied to communicate effectively. Don't be surprised to find that: They could be absent-minded or forgetful.	 Monitor Evaluator Provides a logical eye, making impartial judgements where required and weighs up the team's options in a dispassionate way. Strengths: Sober, strategic and discerning. Sees all options and judges accurately. Allowable weaknesses: Sometimes lacks the drive and ability to inspire others and can be overly critical. Don't be surprised to find that: They could be slow to come to decisions.	 Specialist Brings in-depth knowledge of a key area to the team. Strengths: Single-minded, self-starting and dedicated. They provide specialist knowledge and skills. Allowable weaknesses: Tends to contribute on a narrow front and can dwell on the technicalities. Don't be surprised to find that: They overload you with information.
 Shaper Provides the necessary drive to ensure that the team keeps moving and does not lose focus or momentum. Strengths: Challenging, dynamic, thrives on pressure. Has the drive and courage to overcome obstacles. Allowable weaknesses: Can be prone to provocation, and may sometimes offend people's feelings. Don't be surprised to find that: They could risk becoming aggressive and bad-humoured in their attempts to get things done.	 Implementer Needed to plan a workable strategy and carry it out as efficiently as possible. Strengths: Practical, reliable, efficient. Turns ideas into actions and organises work that needs to be done. Allowable weaknesses: Can be a bit inflexible and slow to respond to new possibilities. Don't be surprised to find that: They might be slow to relinquish their plans in favour of positive changes.	 Completer Finisher Most effectively used at the end of tasks to polish and scrutinise the work for errors, subjecting it to the highest standards of quality control. Strengths: Painstaking, conscientious, anxious. Searches out errors. Polishes and perfects. Allowable weaknesses: Can be inclined to worry unduly, and reluctant to delegate. Don't be surprised to find that: They could be accused of taking their perfectionism to extremes.

Team Diversity - Watching Ourselves!

- Premise: Diversity === Performance
- Supporting Evidence
 - Diverse groups tend to be anti-groupthink
 - Diversity tends to promote resilience, creativity, and adaptability
 - Diversity isn't just background, it's ability (ex. Enthusiastic juniors)
- Rejecting Evidence
 - Poison from those against this due to politics
 - Sensitivity to slights and persecution often lead to real problems (this actually suggests a psych safety concern really!)

Whatever We Think - Avoid the Dominance Hierarchy!

- A specialist team may get into a dominance hierarchy approach with coding on top and docs or QA at the bottom.
 - This is a very non-team way of thinking, so don't do it!

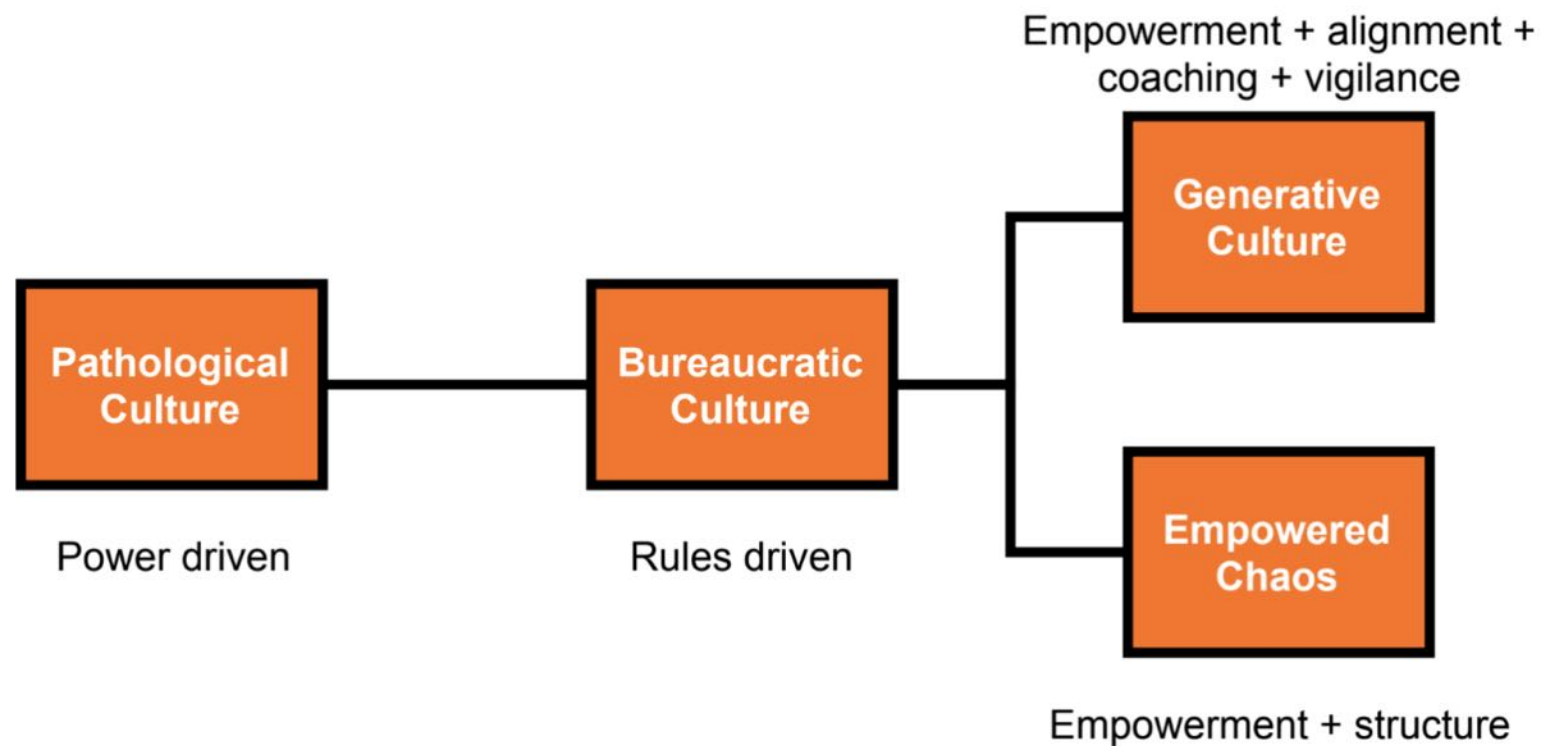


You might see your biases and your thoughts about what is most important in dev being reflected in your reaction to the last few slides. Remember it is not possible to change a bias until you admit to a bias. Is UI design lesser in importance or just something you aren't feeling skilled in?

Designing Better Teams?

Organizational Cultures

The Modern Spectrum of Organizational Cultures



Org Culture Types

Pathological <i>Power-oriented</i>	Bureaucratic <i>Rule-oriented</i>	Generative <i>Performance-oriented</i>
Low cooperation	Modest cooperation	High cooperation
Messengers shot	Messengers neglected	Messengers trained
Responsibilities shirked	Narrow responsibilities	Risks are shared
Bridging discouraged	Bridging tolerated	Bridging encouraged
Failure leads to scapegoating	Failure leads to justice	Failure leads to inquiry
Novelty crushed	Novelty leads to problems	Novelty implemented

Pathological Can Be Good ?

While the pathological culture seems quite problematic in reality many very successful organizations have used it to great effect. Often times this culture is a dog-eat-dog or highly competitive organization with some “alpha” or “silverback” type leaders. In some situations such leaders and style can produce extreme results and if the leaders using this pathological style shower their troops with enough “loot” they may succeed even if people generally are disloyal or dissatisfied.

I added this as it was avoided, but it is my experience true

Bureaucracy Can Be Good ?

We should emphasize that bureaucracy is not necessarily bad. As Mark Schwartz points out in *The Art of Business Value*, the goal of bureaucracy is to “ensure fairness by applying rules to administrative behavior. The rules would be the same for all cases —no one would receive preferential or discriminatory treatment. Not only that, but the rules would represent the best products of the accumulated knowledge of the organization: Formulated by 69 bureaucrats who were experts in their fields, the rules would impose efficient structures and processes while guaranteeing fairness and eliminating arbitrariness” (Schwartz 2016, p. 56).

Generative Can Be Bad ?

Not every generative style can work well. For example, if too much autonomy is heaped on large groups of novice practitioners you may see morale satisfaction at first, but lots of chaos as well. If the chaos is not curbed and begins to effect outcomes the generative organization style starts to mirror the quirky dotCom that burns cash, feels self-important, but really never gets anywhere. This is more common than many would have you believed and the fun and pay is often used to gloss over the actual failure of the organizational approach.

I added this as it was implied by diagrams but not well discussed, but it is my experience true

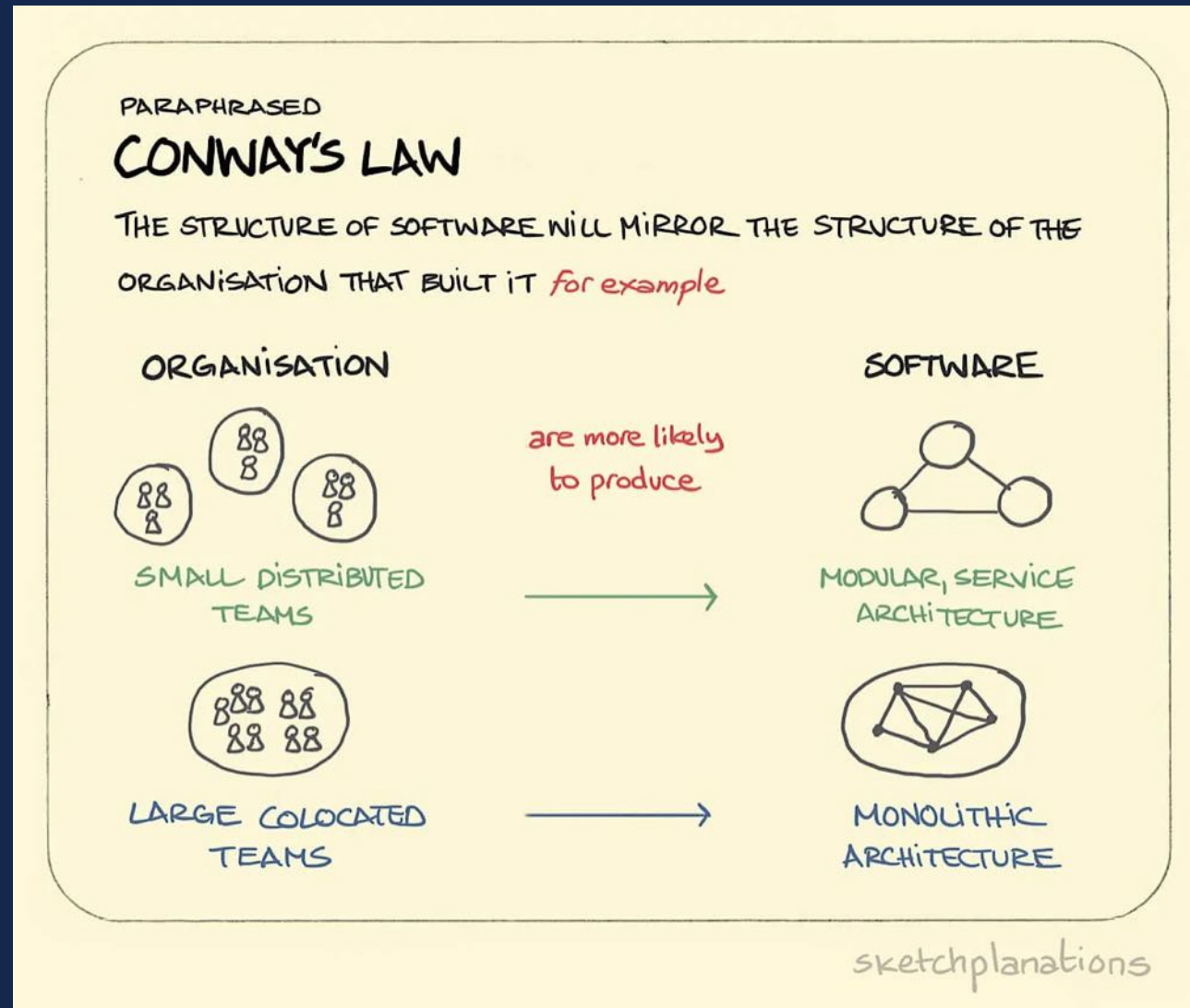
Understanding Team/Org Culture

- Three approaches to modeling organization culture (Schein 1985):
 - *Least Visible: Basic assumptions* - “things we just know”
 - What people think of when you say culture - **Values** - “collective values”
 - *Most Visible: Artifacts* - “mission statement, rules, etc.”

Teams that live long pass values on, perform their “ceremonies” consistently and have many clear, available and put to date artifacts!

Conway's Law Preview

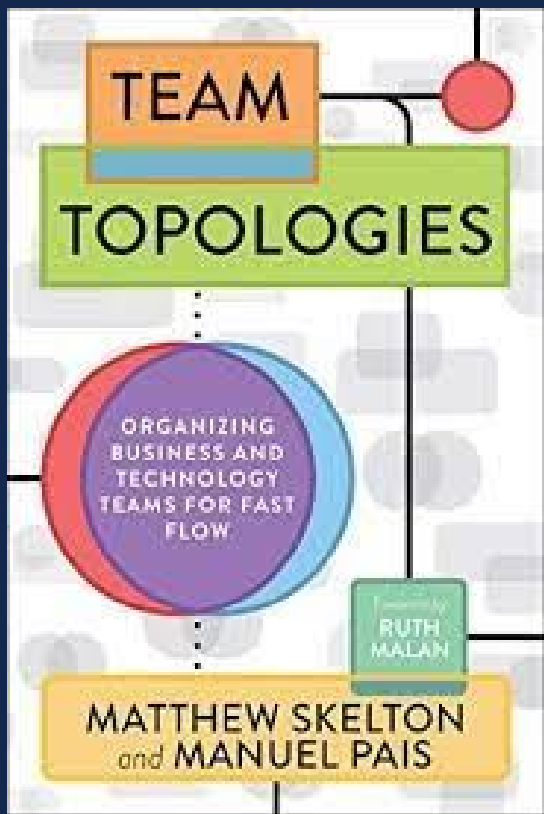
- I agree with this to a point, but it also in my mind has certain HR problems about it in that to change architecture often means to dramatically reorganize the organization. Sometimes that change especially if it is promotion focused has a perverse effect (note a current Google example)



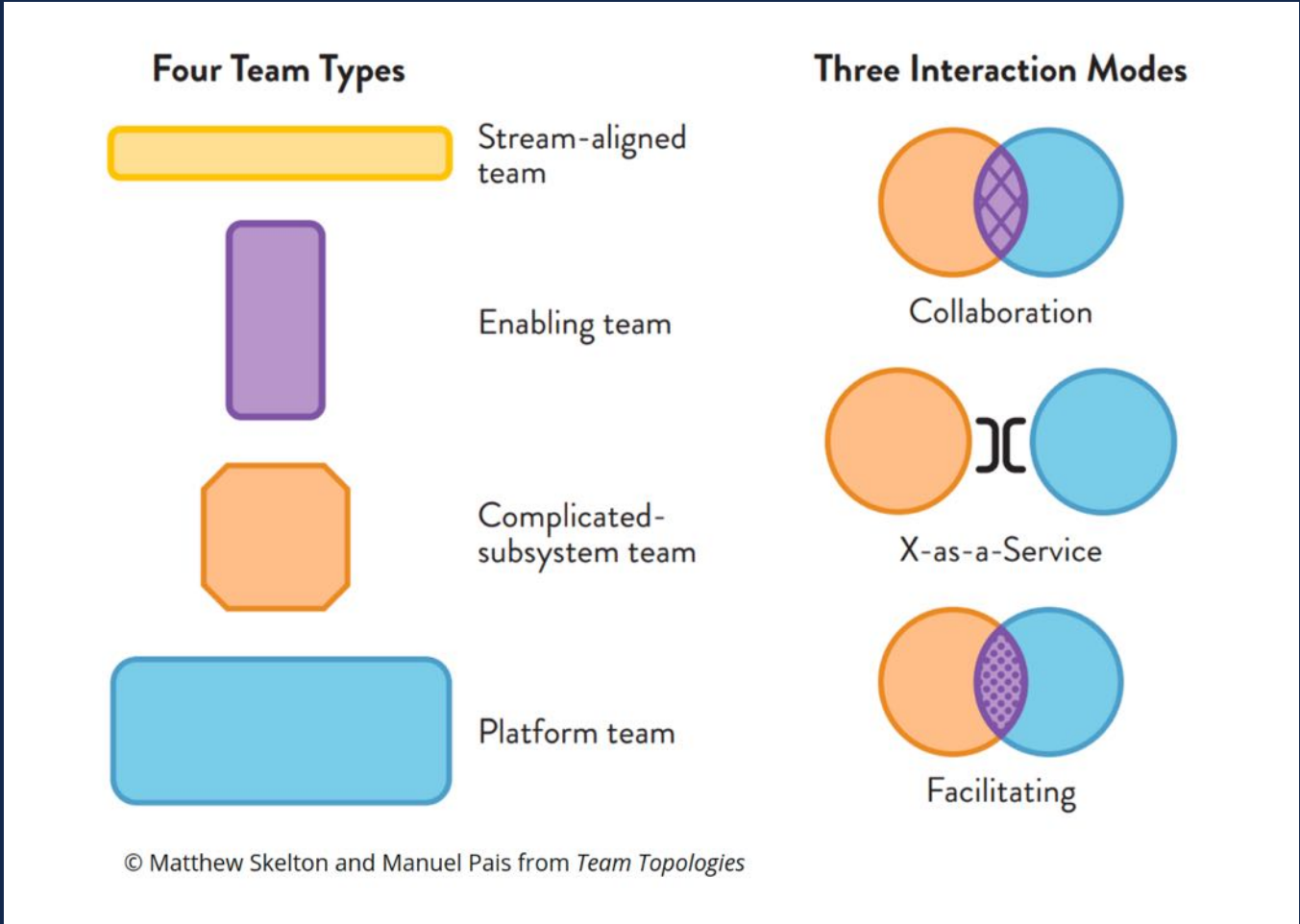
Team Formations

(Quite Useful for FAANG and Big Tech Bound Folks)

Team Topologies (Common in FAANG)



<https://teampatterns.com/>



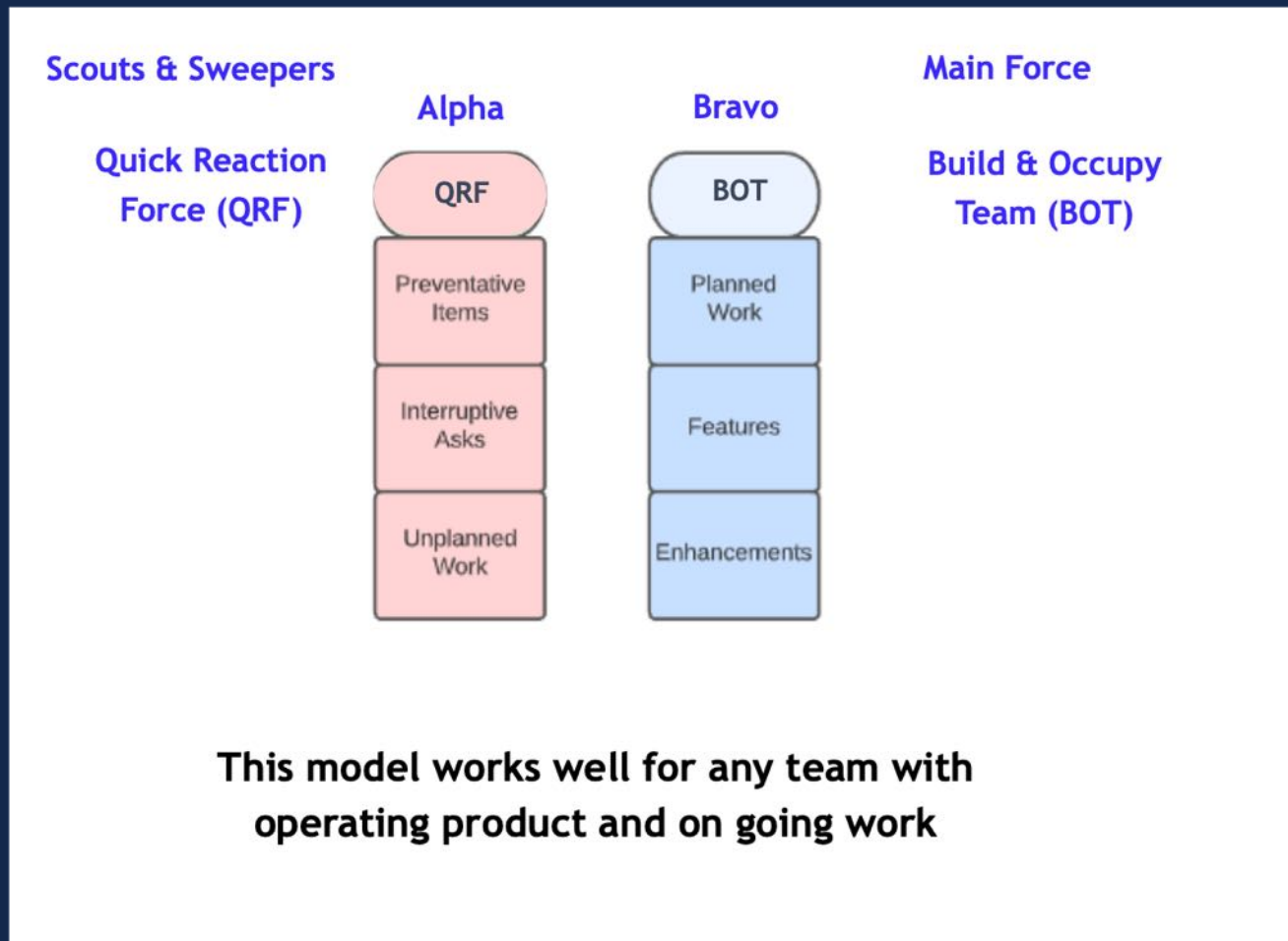
Team Topology Insights

- Small teams = More Trust
 - Beware of trust boundaries in teams of teams (silos)
- Long lived teams become elite teams
- Elite teams are filled with team first members
- Good teams have low team cognitive load, people should have simple directions and confidence in their knowledge
 - If simplification need an enabling team needs to get involved
- Account for the friction of handoffs within and between teams
- Respect Conway's Law (try a reverse Conway if needed)
 - Socio-technical architecture is a thing!

Addressing Interruptions at Team Level

- The nature of the work and current operational conditions should be acknowledged and the team structure define to reflect that
- Commonly in organizations that ship a system, there are the day to day crisis, growth issues, and other interrupt driven events - such situations need a **Quick Reaction Force (QRF)**
- The rapid shifting and mitigation solutions of a QRF as well as the lack of large free time chunks can stall development so a **Build Occupy Team (BOT)** needs to focus on what it means to retain value over time

Visualization of the Team Split



Source: <https://betterprogramming.pub/engineering-org-structures-the-qrf-team-model-7b92031db33c>

Challenges of QRF and BOT Teams

- Inherently some people may thrive on QRF work while others may be quickly burned out by it
- Those on a BOT team might need to put time on QRF to understand the “real world” while QRF members might need to be more broad thinking to understand decisions made
- The most toxic aspect of the team split is that in some cultures it might suggest an A and B team split with a ranking. This can come even in the names - Alpha and Bravo for example

Implementation Details

- QRF needs generalists, BOT needs specialists
- QRF needs pragmatic, flexible thinkers who are action oriented, BOT might need allow for some more idealistic and aspirational individuals who want space to think
- QRF may not need the size to be effective. It might be 80/20 BOT/QRF in terms of head count.

Implementation Details

- QRF likely needs to be driven by self-starting volunteers and culture should be built showing that it is a honor to be good enough to be on such a team.
- Putting your green recruits on QRF is very risky, though admittedly some may rise to the occasion
- BOT teams need to understand their ability to have “time and space” to do their work is made possible by the success of QRF because if they fail it is likely “all hands on deck”

QRF and DevOps

- It would seem as if QRF was just a different name for DevOps. There is some truth to this, but a main differentiation here is that this team is doing dev work more than just operating work.
- DevOps teams often are more operational while a QRF team is versed in the system beyond just operating it.

End FAANG BigTech Info

Culture and Teams

Organizational Culture and Team Effects

- Organizational culture studies are not new many cite Westrum (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC1765804/>) as setting a tone for how we think about org theory
- Westrum's org theory posits that **orgs with better info flow function more effectively and culture strongly influences such info flow. In short good culture = more info flow and finally better results.**
- Good culture requires
 - Trust
 - Collaboration
 - Transparency
 - Acceptance of failure and avoidance of blame

Question: Culture is...

- Mostly Emergent
- Mostly defined / designed
- Initially defined, but ultimately emergent



Team Trust

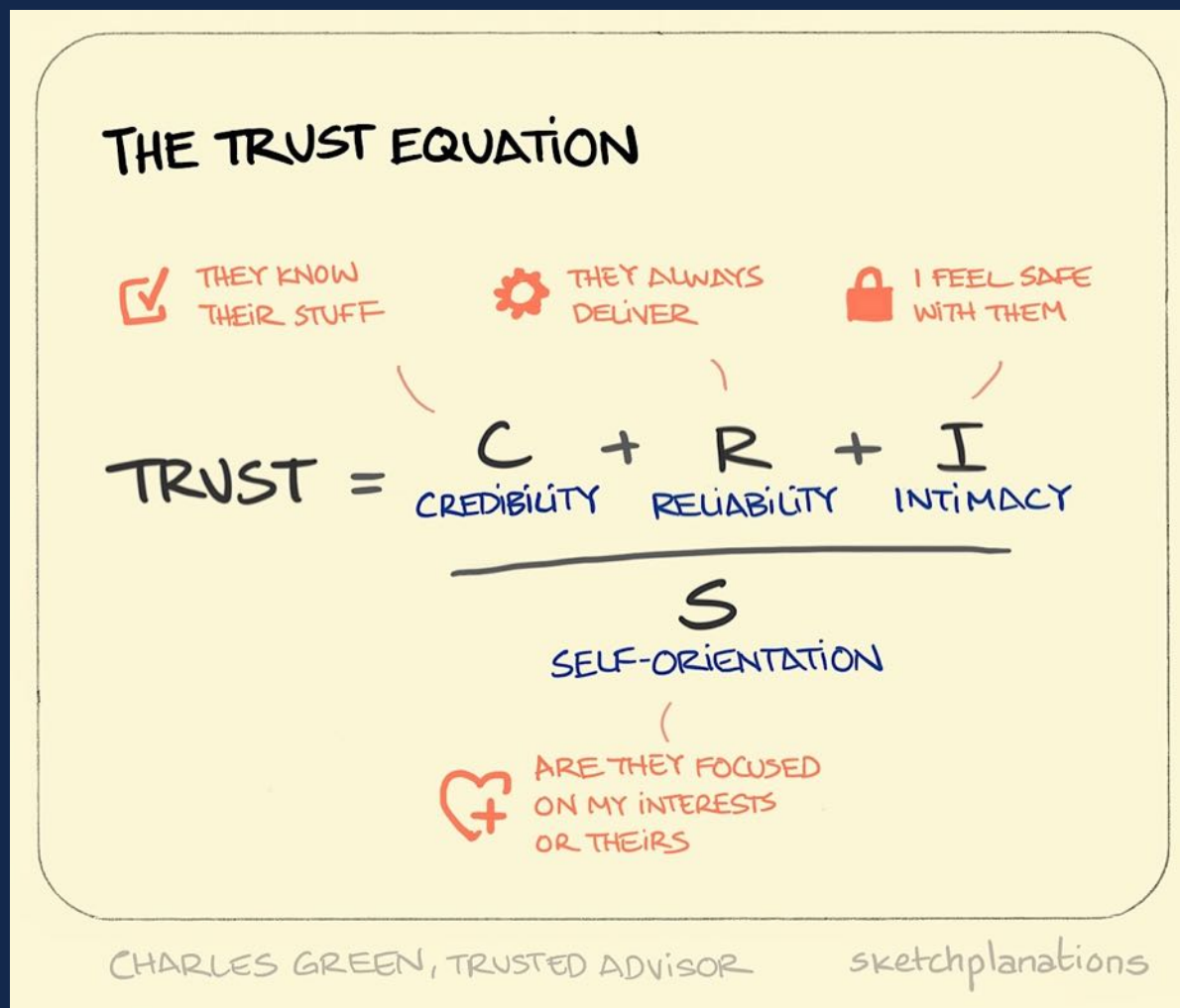


TRUST is Built on



FIRST IMPRESSIONS

Seriously a Group Trust Equation



WHAT TRAITS DO GREAT MENTORS, PROJECT LEADS, TAS IN CSE 110 HAVE?

1. Is a good coach
2. Empowers team and does not micromanage
3. **Creates an inclusive team environment, showing concern for success and well-being**
4. Is productive and results-oriented
5. Is a good communicator — listens and shares information
6. **Supports** individual **development and discusses performance**
7. Has a clear vision/strategy for the team
8. Has key technical skills to help advise the team
9. **Collaborates across** the class
10. **Is a strong decision maker**

PROJECT OXYGEN

WHAT TRAITS DO GREAT MANAGERS AT GOOGLE HAVE?

1. Is a good coach
2. Empowers team and does not micromanage
3. **Creates an inclusive team environment, showing concern for success and well-being**
4. Is productive and results-oriented
5. Is a good communicator — listens and shares information
6. **Supports career development and discusses performance**
7. Has a clear vision/strategy for the team
8. Has key technical skills to help advise the team
9. **Collaborates across Google**
10. **Is a strong decision maker**

The Perfect Team

What Google Learned From Its Quest to Build the Perfect Team

New research reveals surprising truths about why some work groups thrive and others falter.

By CHARLES DUHIGG

FEB. 25, 2016



https://www.nytimes.com/2016/02/28/magazine/what-google-learned-from-its-quest-to-build-the-perfect-team.html?_r=0

Results: 5 Things For Great Teams

1. Psychological Safety
2. Dependability
3. Structure and Clarity
4. Meaning of Work
5. Impact of Work

BUT... #1 apparently is paramount for the other 4

Psychological Safety

- Does the team have comfort taking risks?
 - Worry, shame, insecurity, embarrassment, etc. are going to really keep you from doing this
- Without this safety people don't own up to mistakes and so problems fester
 - Same worries as above, admitting weakness is a "risk"

Psych Safety

- Three cultural dimensions are critical for team resilience
 1. **Integrity** - ethics with candor and without retaliation
 2. **Innovation** - fearless collaborative creativity with shared success
 3. **Inclusion** - authentic membership and respect

Source: <https://hbr.org/2022/08/resilient-organizations-make-psychological-safety-a-strategic-priority>

Effects of Psych Safety Reduction

- **Group think** (often a side-effect from domineering personalities esp in leadership roles or creative process esp if coupled with ridicule or retaliation)
- **Project risk and quality reduction** (unwillingness or hesitation sharing opinions often for same reason as group think)
- **Poor retention** (the equation of \$ does not eliminate other satisfaction requirements for people, worth discussing massive pay disparities)

Building Psych Safety

Four general parts

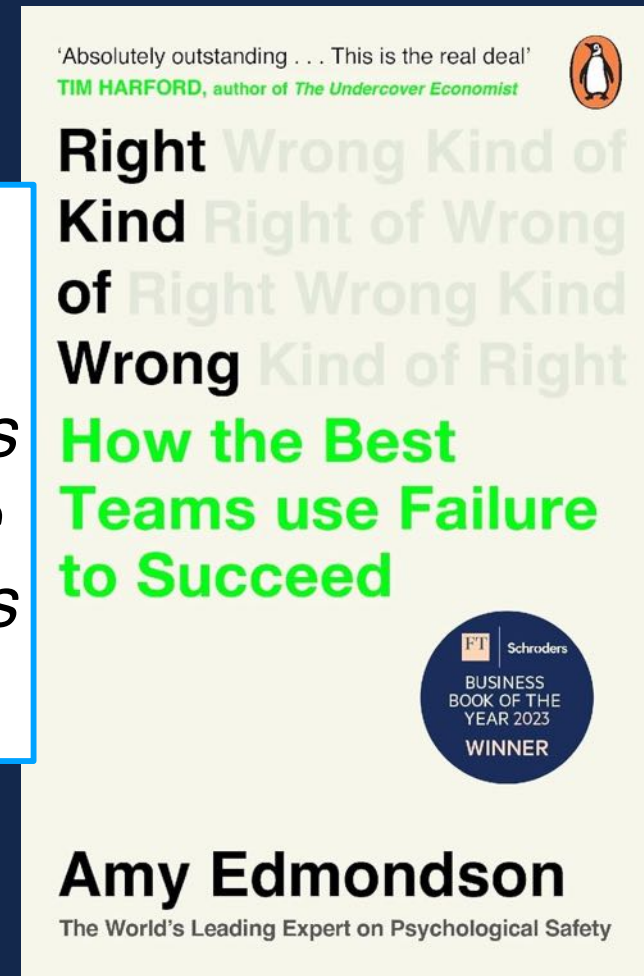
1. Model
2. Allow Failure
3. Avoid Blame
4. Empathize

It will take time and comfort obviously thus our team building efforts, various onboarding bootcamps, emphasis on “culture” at various companies, mixers, etc.

TL;DR - Q: The Secret to Group Success?

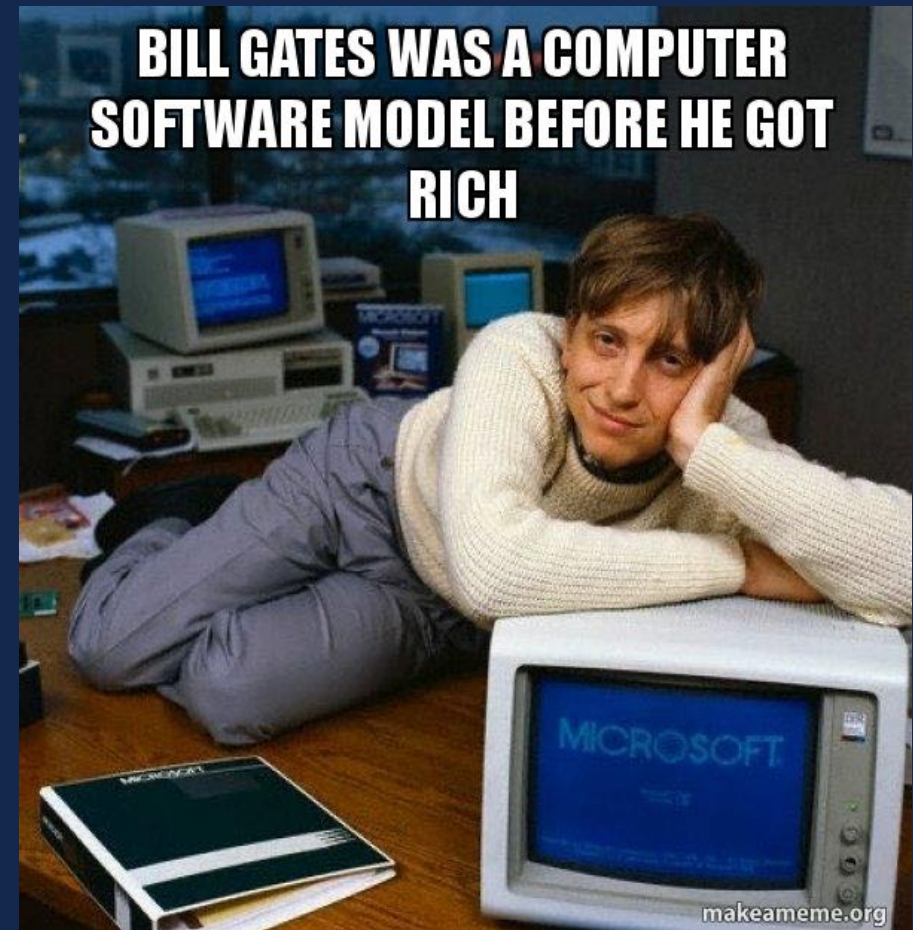
A: Being Wrong Correctly

“The best teams didn’t make more errors; they admitted more to making errors. Dysfunctional teams admitted to very few, for the simple reason that nobody on those teams felt safe owning up.”



Modelling

- Try to show and do as opposed to tell
 - Leader: “I command everyone to be nice to each other!”
 - Yeah, that won’t work
 - “Everybody is being such a jerk right now!”
 - Yeah... you too
 - Instead...
- Be a role model yourself
- Leaders can have biggest impact, but all team members can do it
- Be humble, vulnerable, willing to make mistakes, etc. and set the tone



“High Fives” and Celebrations

- Highly functioning teams share victories properly
- The “striker” doesn’t take all the credit as everyone shares in the victory
- In dev work this means acknowledging your teammates in chat, meetings, etc. for the things they do and how they support the effort



Shared Defeats

- A well functioning team doesn't solely blame the players who might have had an off day or "dropped the ball"
- The loss should be shared by all
- Blaming and sniping happens when the deal is fracturing



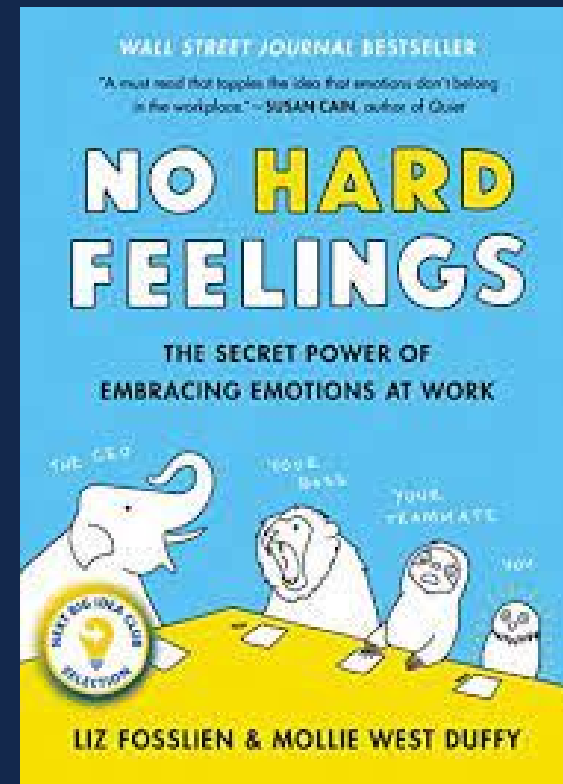
Beware of cliques

- Be careful with teams within teams – aka cliques
- Teams in this class especially with many previous experienced people who know each other can form cliques easily
- When a team works together very often it is quite possible it becomes clique-ish so that new team members can't get involved deeply
 - Aside Negative: The continued challenge of 'brogrammer' and 'gender problems' in our industry have been significantly amplified by such clique cultures. Watch out for "fit" being used as exclusion in hiring or work allocation!
 - Aside Positive: close working relationships can create efficiency that may difficult without clear group inclusion

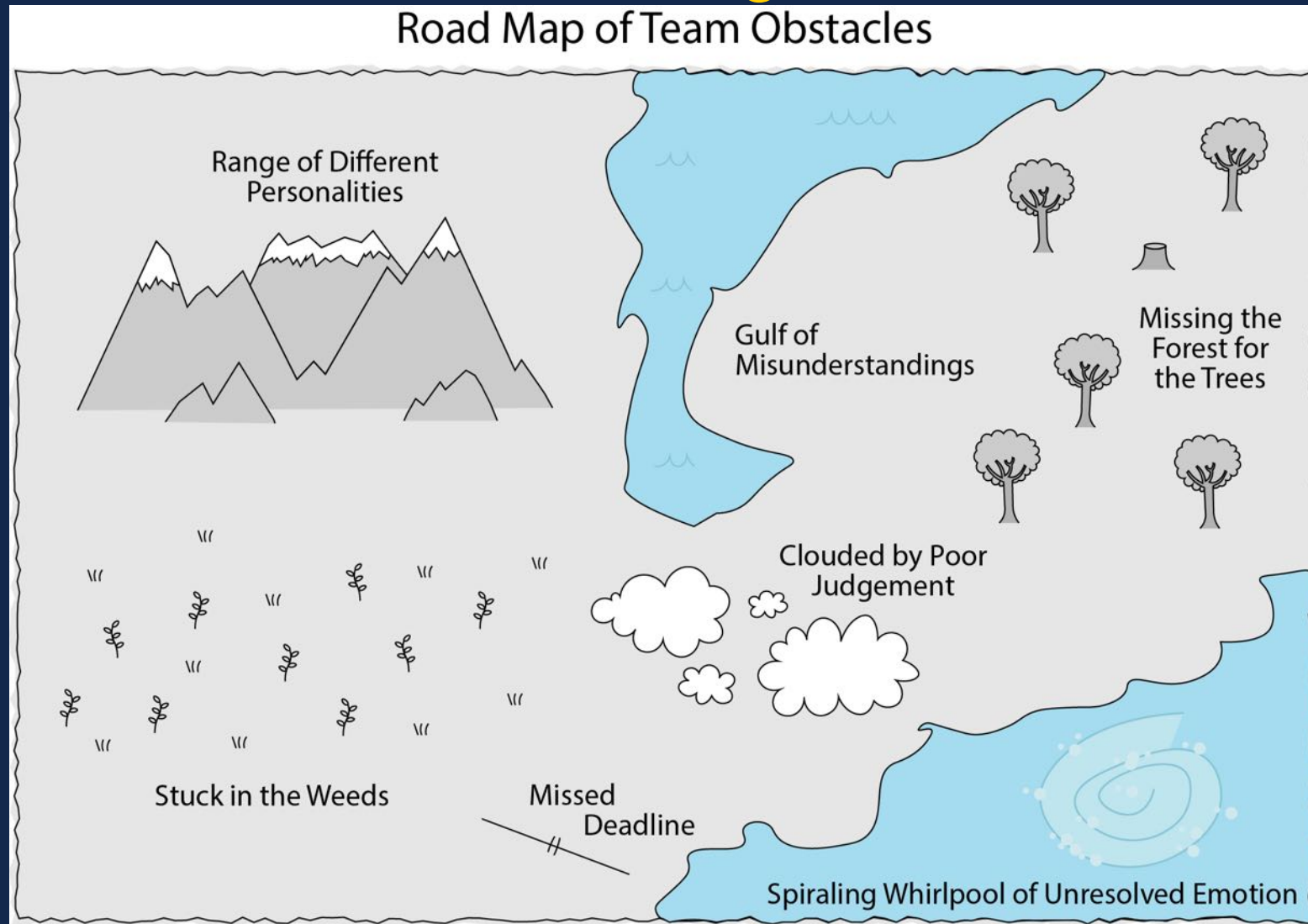
Group Communication Part 2

Feelings for the Teach Win?

- There appear to be more and more works addressing the need for improved team communication
- Some of the ideas in the next few slides come straight from works like *No Hard Feelings*



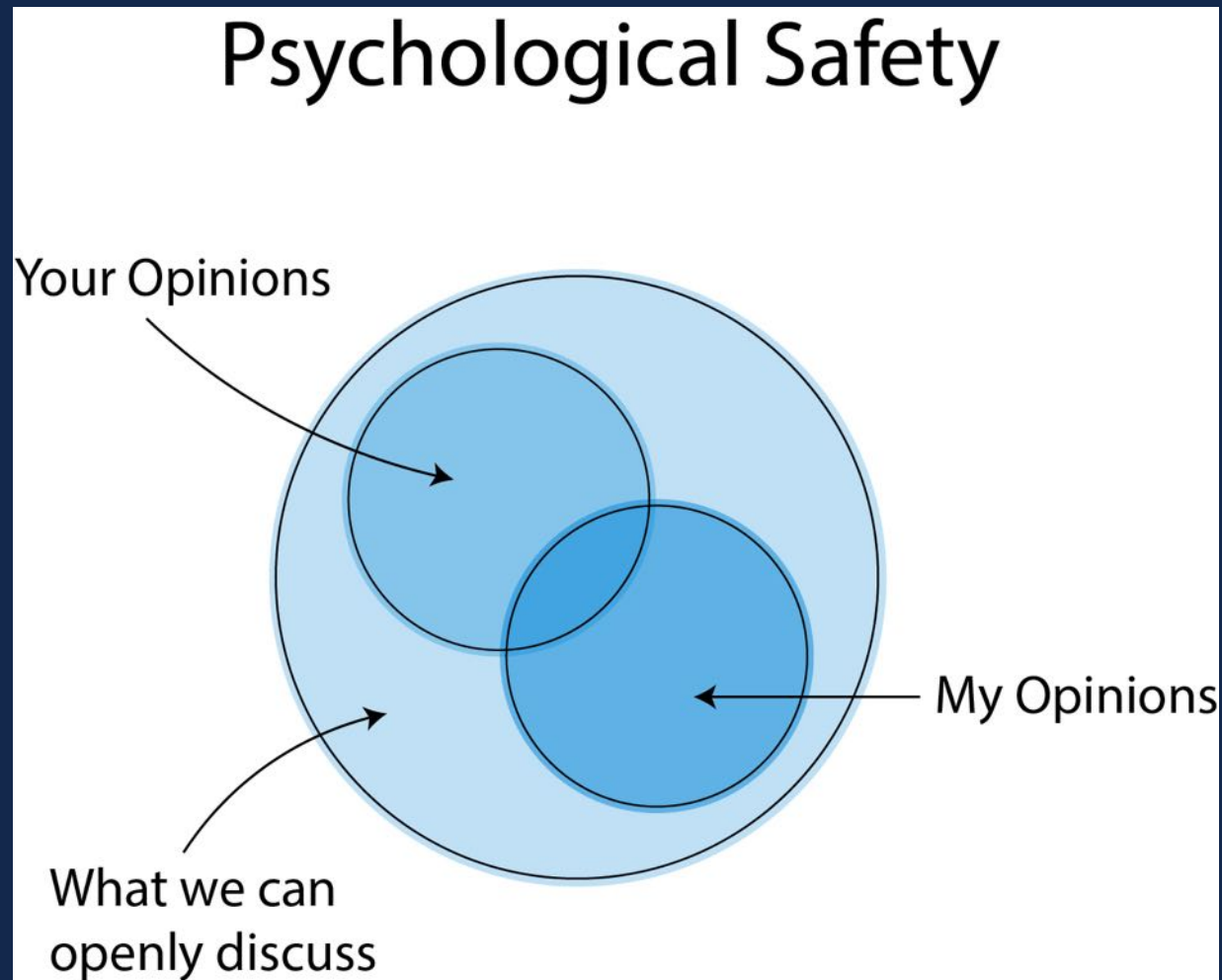
Good Comms Can Be Tough!



Communication Patterns

- Given we have a diverse set of team members (different experiences, cultures, genders, and more) we need to be mindful we also likely have different communication patterns
- Sometimes we aren't aware of our patterns and we need to give people a benefit of the doubt if they are exhibiting patterns that might not work for you — it may not be understood by the other person
- What we consider as not a big deal, very often is a big deal to others. A team with psych safety can deal with this!

Psych Safety and Communication



Yet a Surprising Example?

Was at a training once where the facilitator said
“Unsolicited advice is criticism. Always.”

Half the room audibly gasped/ objected.

The other half shouted a chorus of yes/thank you/
amen.

She offered no quarter to the “just being helpful”
brigade. It was glorious.

8:50 AM · Sep 17, 2022 · Twitter for iPhone

1,806 Retweets **215** Quote Tweets **6,950** Likes

Objections and Advice is Required (when appropriate and when delivered correctly)

	Positive Intent	Negative Intent
Willing to Raise Objections	Ideal Teammate	Dissenter
Unwilling to Raise Objections	Sycophant	Silent Grump

Avoid Communication Domination



Avoid Drama Triangles

- Given varying personalities it is possible to have some team drama in communication
- Be careful of judging the behavior of people as an innate characteristic rather than just a way they might be acting.
- Consider what we might consider negative behaviors can be modified and become positive behaviors



Try to Separate Problems from People



Love and Hate the Game Not the Player

	I Like You	I Hate You
I Like Your Idea	No Conflict	Relationship Conflict
I Hate Your Idea	Task Conflict	Feud

Communication Practice Makes Better

- Start small and grow
- There is reason for our “warm-up”
- Be careful not to hold you or your teammates to some standard of communication perfection
 - Be Digital (t/f) on message, Analog (a range) on effectiveness and tone

Admission - Fails may happen just keep them small!

The Key to Success is Failure?

Failure is an option

- You must remove the fear of failure
- Change goal to a focus on learning and experience
- Of course, when you do be prepared. Failures will happen!



All aboard the Failboat

**I DON'T ALWAYS
BREAK THE BUILD**



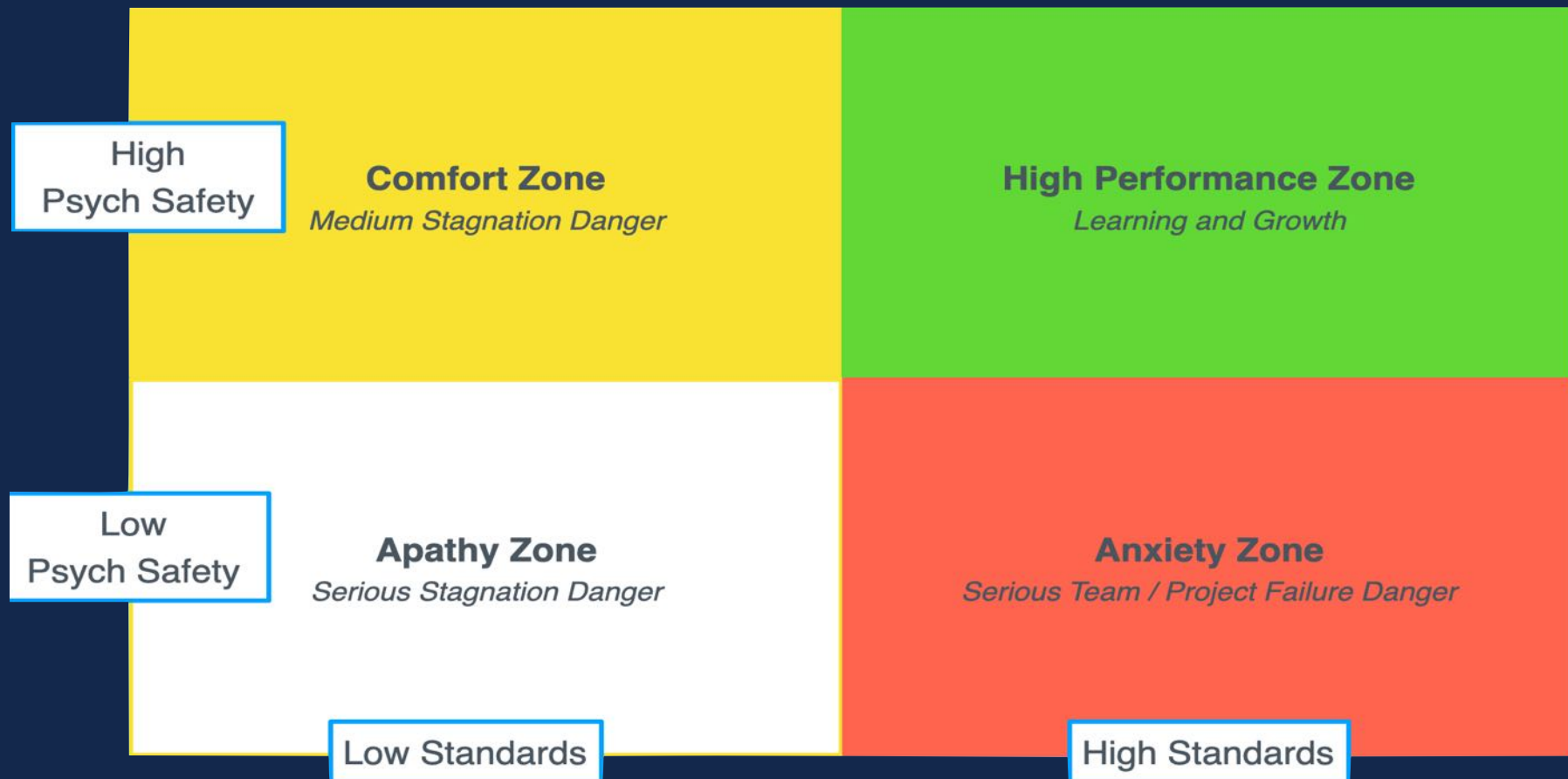
**BUT WHEN I DO,
I BLAME IT ON YOU**

Get Off the Blame Train!

- Stuff happens get over it!
- Remember a 'pass' given to a teammate might be redeemed in the future for your mistake if you don't blame them now
- If you allow the blame to happen too much, then people start to fear revealing failure and psych safety starts to go away
- We really need to have some compassion & empathy for our teammates

Standards and Safety Matrix

Combine two good things create a danger?





EMPATHY

would this help?

Empathize

- Everybody is different
- Try to “walk in their shoes”
- Remember nobody* is trying to be stupid, evil, etc.
 - Yes there are certain toxic (or even sick/criminal) people but we are not going to let an exception spoil the norm
 - Beware without empathy toxicity creeps in and with that teams can ‘die’

Empathy cat



Wants to walk in ur shoes

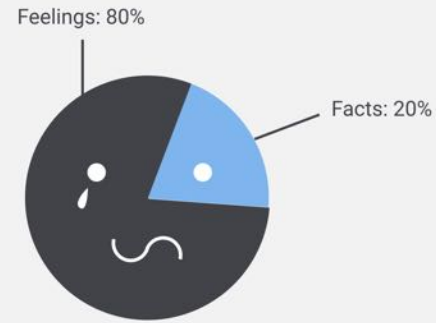
Logic or Emotion?



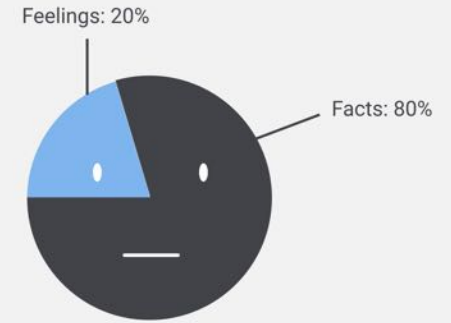
Spock



Half & Half



Emotional

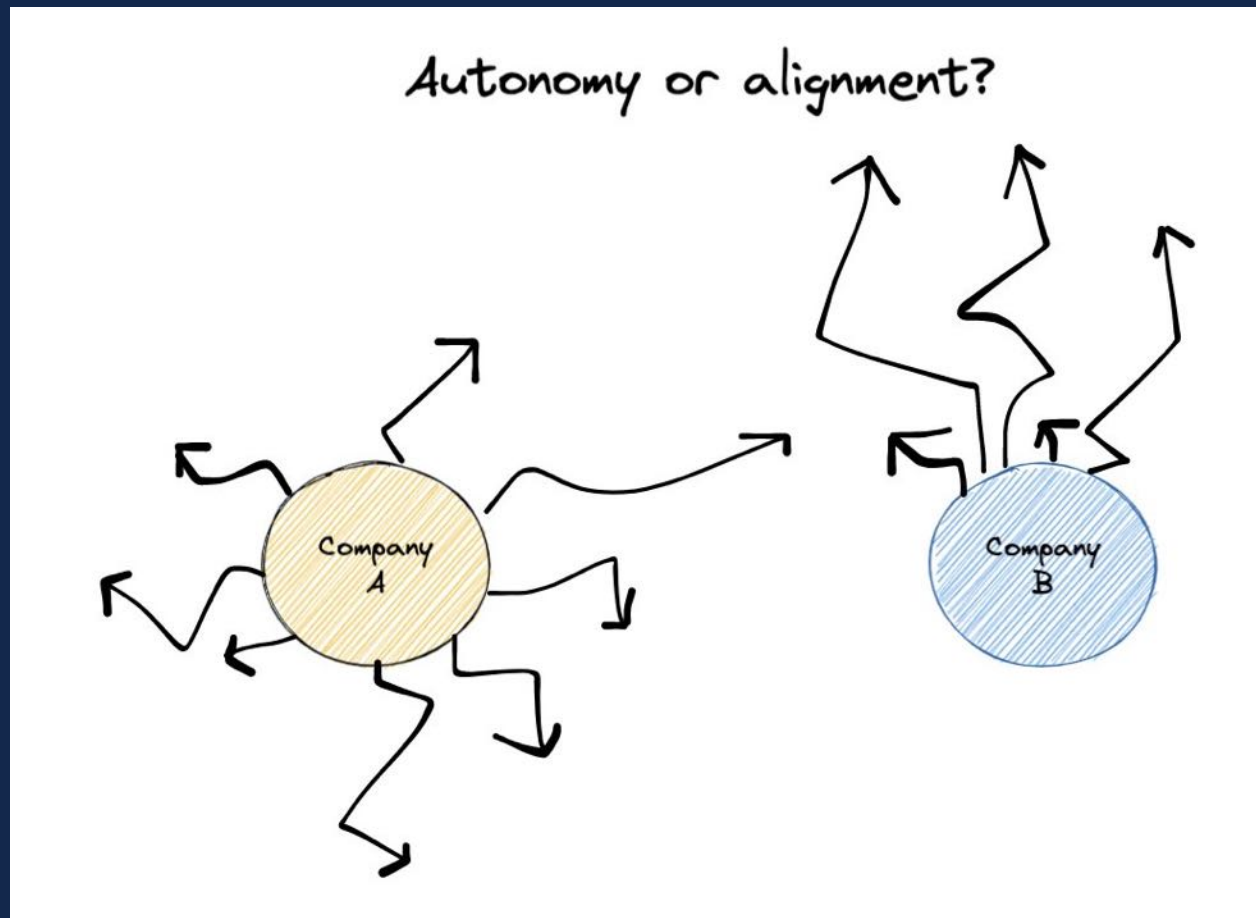


Logical

The other 4 points

- Dependability
 - Remember the skip the meetings problem!
- Structure and Clarity?
 - What happens if we don't have that from life?
 - Who then gives structure and clarity?
- Meaning and Impact of work
 - “But I’m just doing the _____” (docs, testing, unit tests, refactoring, etc.)
 - Kind of a bad attitude
 - “Coach I don’t want to play on the team unless I’m the striker!”
 - Shouldn’t you be like the [NASA janitor](#)?
 - JFK to Janitor asking about what he does
 - Janitor: “Well, Mr. President,” the janitor responded, “I’m helping put a man on the moon.”
 - Not every project or app is a game changer (or is it to someone at least?)

Which is more important - autonomy or alignment?



Alignment Thinking for Groups: Act Like a Tree



Leaf Decisions: Make them, act on it. I don't have to know about them. They are usually reversible.

Branch Decisions: Make them, act on it. Let me know about them and share your heuristics so that we can learn. Reversible with some effort.

Trunk Decisions: share your decision and heuristic before acting on it. Get some feedback from me and a peer. These decisions will have broader company impact.

Root Decisions: We will make the decision jointly. These are decisions that when poorly made can cause major harm to the company.

Susan Scott's "Fierce Conversations," offers the Decision Tree alignment model

Smells of Teamy-ness

- Difficult to pin down like what is a “friend”, we know when it is or isn’t but there are signs. Google’s data did point out a few things and I added a few of my own *
- A Group Intelligence and Ownership emerges
- Use of “we” over “I” *
- Small member domination is lesser (more even number of communication points)
- People don’t tend to talk over or ”at”* each other
- There are shared views
- Interestingly Google seemed to find that have a better gender density correlated with team success in their study as well.

Group Patterns, Procedures, and Ceremonies

- As developers we tend to seek patterns, procedures, and ceremonies that reinforce success
 - There is nothing wrong with this effort and having structure is a very important aspect of a healthy group
- But...do not confuse your awareness of a particular pattern, procedure or ceremony and its presence during a successful project as a direct correlation
- Looking at the literature we see literally hundreds of patterns and tips for group behavior in the Agile community. Many are just slightly different than another. **TL;DR - ceremonies and patterns can't correct a group!**

See spreadsheet linked for example of 200+ procedures and approaches

<https://drive.google.com/file/d/1vRi1YmWAH7EGALPz6jWv4jrDY6ozp5BA/view?usp=sharing>

More On Team Smells

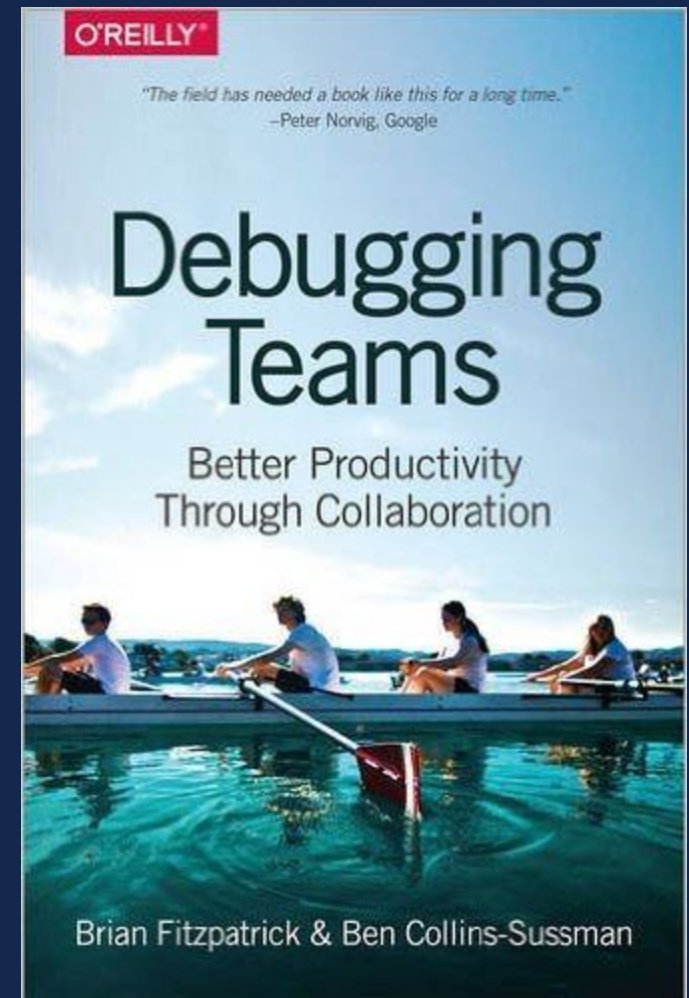
- What smells good and bad should we look for?
- What questions should we ask?
- Ideas
 - Is the team “playing” well together?
 - What’s the “score”? Do we keep score?
 - Are we winning or losing overall?
 - Are we improving?
 - Does somebody need to sub?
 - Should someone change positions?
 - This might lead to rest or swap even?
- How do we measure “teamy-ness”
 - Effort? Enthusiasm? Ability? Quality?

Very Specific Agile Team Anti-Patterns

- Letting the leader or product owner rather than the team solely design the HOW of a solution as opposed to sticking with the WHAT of the problem
- Believing More Features = Success
- Believing More Story Points = Progress
- Assuming a successful sprint is solely meeting all tasks
- Lacking a coherent and shared product vision
- Attempting to please every possible user or stakeholder
- Not allowing appropriate* conflict

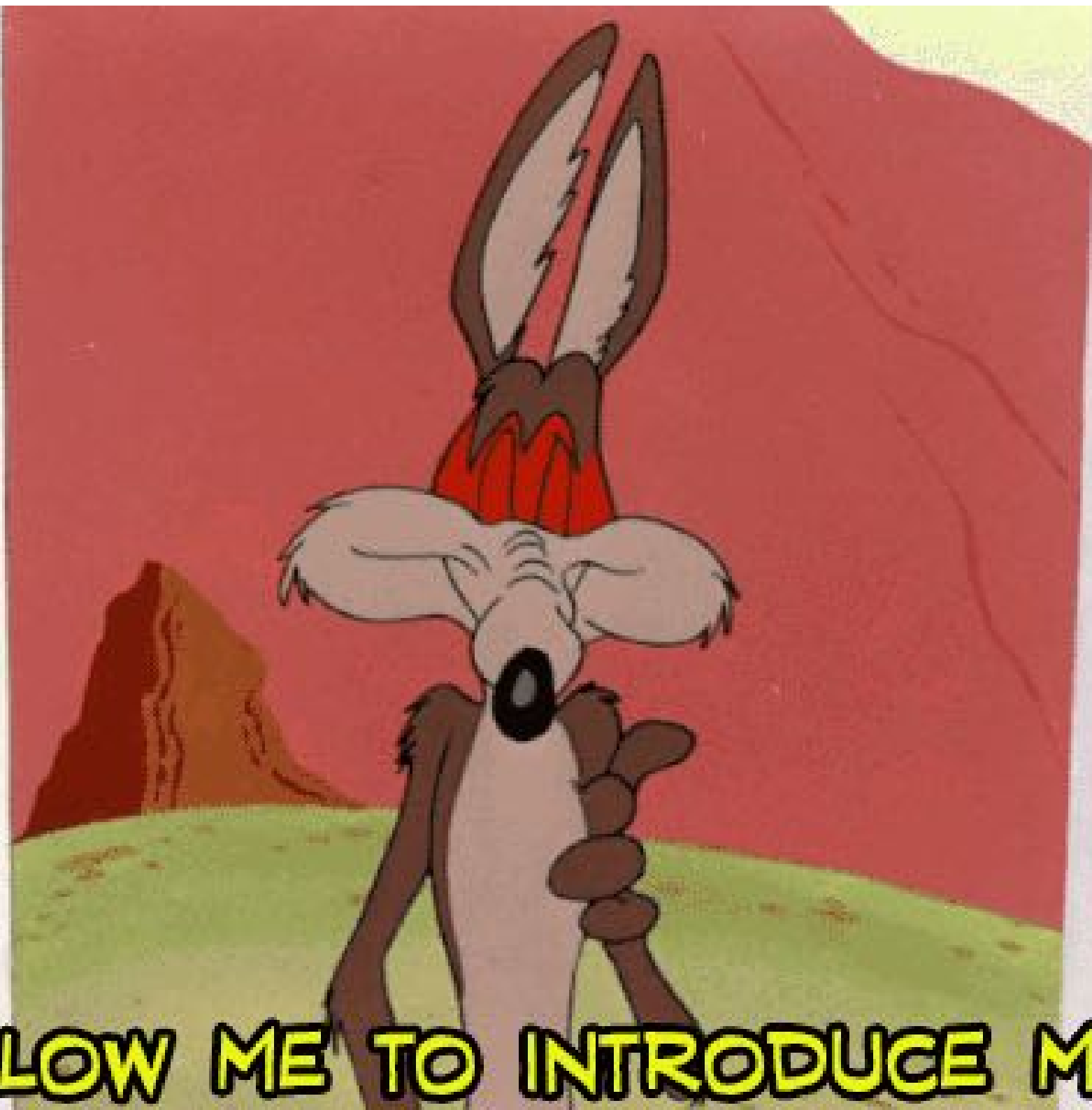
Debugging Teams

- If you believe some of the premises from last lecture the secret weapon a software engineer needs to master is often softer than code!
 - Much of the content out there focuses on programmer self-improvement. Following Postel's Law for yourself that makes sense!
 - Today's content tries to address team issues, though it starts of course with you again
- These slides represent some of the findings from *Debugging Teams* by Brian Fitzpatrick & Ben Collins-Sussman with numerous items from other books like *Joel on Software* and the Prof's experience mixed in for good measure



Geniuses

- A lot of our problem in groups starts with ourselves as we saw last time. Let's start with the rough root cause of many problems for programmers
- Subscribing to the Myth of the Genius Programmer
- Makes sense in a perverse way if we really equate our self-worth dominantly with what we code
- Unfortunately, the emphasis on self is a myth as even recognized geniuses are nothing without a team
 - Jordan, Jobs, Gates, Torvalds, Musk, etc. without their team(s) they are not nearly as powerful as with them



ALLOW ME TO INTRODUCE MYSELF

FAANG Fail: Rarely If Ever Do This



Caligula's Hoss @moonpolysoft · Mar 14



Quick tip for all software engineers: a great way to punch up any meeting and make your coworkers admire you is to drop a "well at Google/Facebook/Netflix we did it this way," everyone absolutely loves that.

74

190

2,938



There is no one true way

≠ The conditions of where X worked at Y might not be the same as current conditions

Watch the subtle implication that you are saying place Y is better

Hiding

- When we subscribe to this apparently noble intent bad things can happen
 - From last lecture this leads to the impostor syndrome in extreme
 - Less extreme leads to code hiding and lack of communication
 - Ex: “I don’t want people to see what is clearly a work in progress, otherwise they might judge me!”
 - Ex: “I am not sure my idea is any good, so I’ll just say nothing”
- Hiding also eventually leads us to information hoarding, which then build us into fake ‘geniuses’ and may lands us job security by information obscurity.
- Our intentions while self protecting end up backfiring on us



Bus Factor

- Bus Factor – A numeric value that represents the number of people on the team/project that if hit by a bus would doom the project
- A low Bus Factor represents extreme project risk!
- Bus Factors often grow with info hiding and hoarding
- Sometimes teams accidentally worsen their bus factor
 - “Jennifer is the best at the build scripts, we will just let her work on it since that is most efficient”
 - “It doesn’t make sense for everybody to do everything, so we will each specialize ourselves”
- This is bad Engineering at the team level – there is no redundancy or fail safe at the human level! Sports, symphonies have back-ups and subs does your team?

Bus Factor Story

Senior Engineer X: I really think I can do better than NodeJS, it will be your PHP in JS idea. Let's do this and build something amazing.

Repeat for 2 years with demos

Me: (after finally deciding to go for it) Ok, let's do it

Senior Engineer X: Awesome! About time, let's do this.

6 months pass, code is created with a bus factor near 1 and baked into systems and then...

Senior Engineer X: I am quitting. I'm bored and this isn't going anywhere.

Result: Project was doomed and looked to never really work in a deep way without significant time/\$ investment. To recover 1 -2 developer year+ effort to unroll in-house code and revert to NodeJS

Bus Factor Visualizer?



<https://github.com/git-truck/git-truck>

Programmer Life Cycle?

New Hires start at step 1

1. Ambitious
2. Productive
3. Irreplaceable → BUS FACTOR RISK!
4. Resentful → QUIT
5. Bored → QUIT
6. Unproductive → FIRED

Is this a microcosm of how some group projects go?

Let's avoid getting past step 2 or so and if we do it makes sense to "bail before we fail"

BTW I am not a fan of this can't we go 3 - Growth, 4 - Mastery, 5 - Transition / Promotion?



You Have to Work With Others

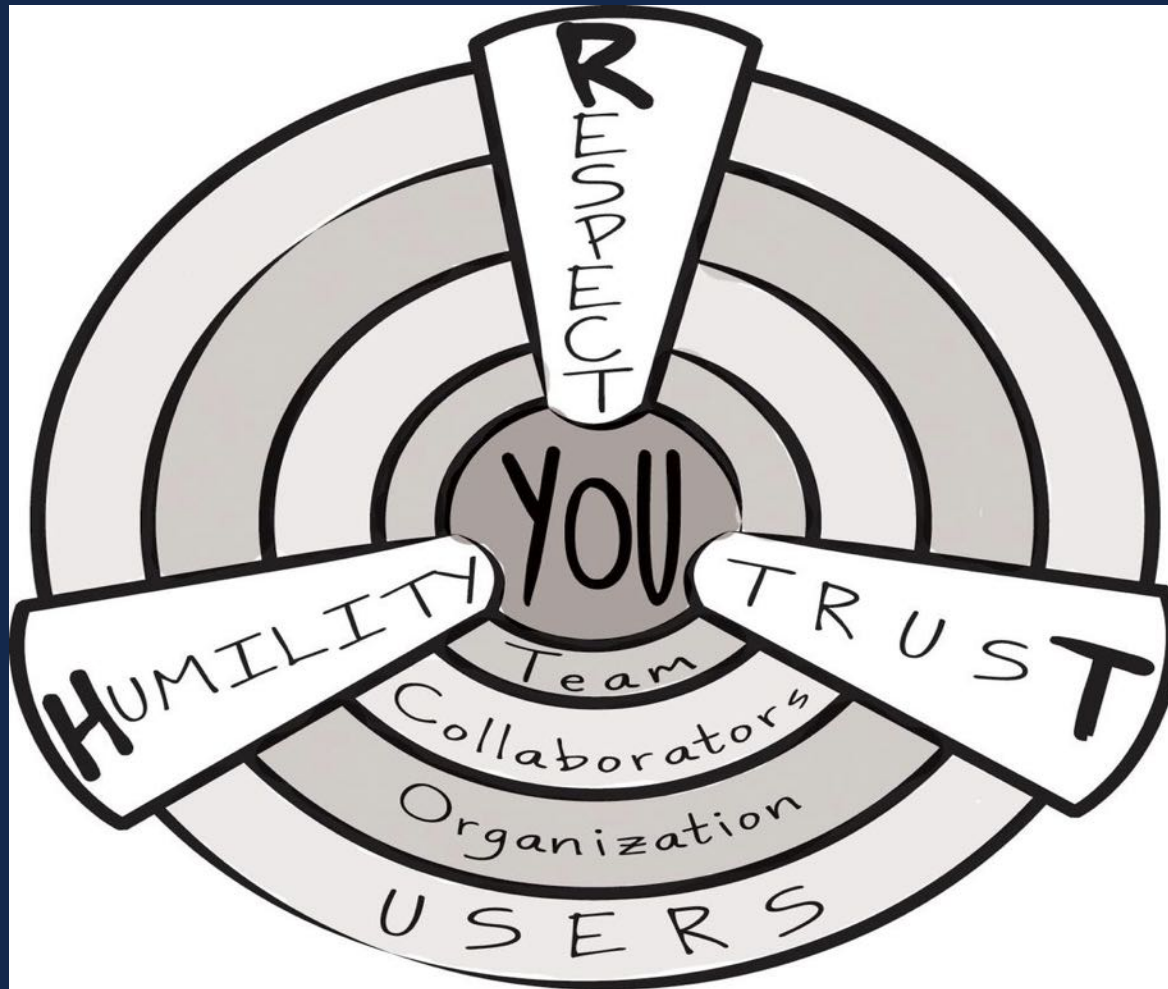
- Any sane organization shouldn't let you become the invaluable hoodie wearing hermit developer
 - Some will but they shouldn't as things can and do go bad
- You can do it on your own clearly, but you shouldn't
- You learn from others which makes you become a better programmer
- You can get more accomplished with a team than by yourself

Working alone is just riskier than working with others!

HRT (HEART) Method

- **Humility** – you aren't perfect, you should be open to self-improvement and growth
- **Respect** – you appreciate others, their intentions and their efforts. You treat them as fellow human beings with feelings just like you
- **Trust** – You trust that others are competent and will do the right thing if you let them

H(umility) R(espect) T(rust)



HRT doesn't solve, it explains

- Social conflicts generally can be traced back to a combination of these three pillars
- Understand that since there are many layers here it requires that everyone acknowledge these pillars otherwise it doesn't do much
 - **Translation:** You can employ this, but others may not, if they don't, they should be ejected from the team, collaborators group, organization, or user population

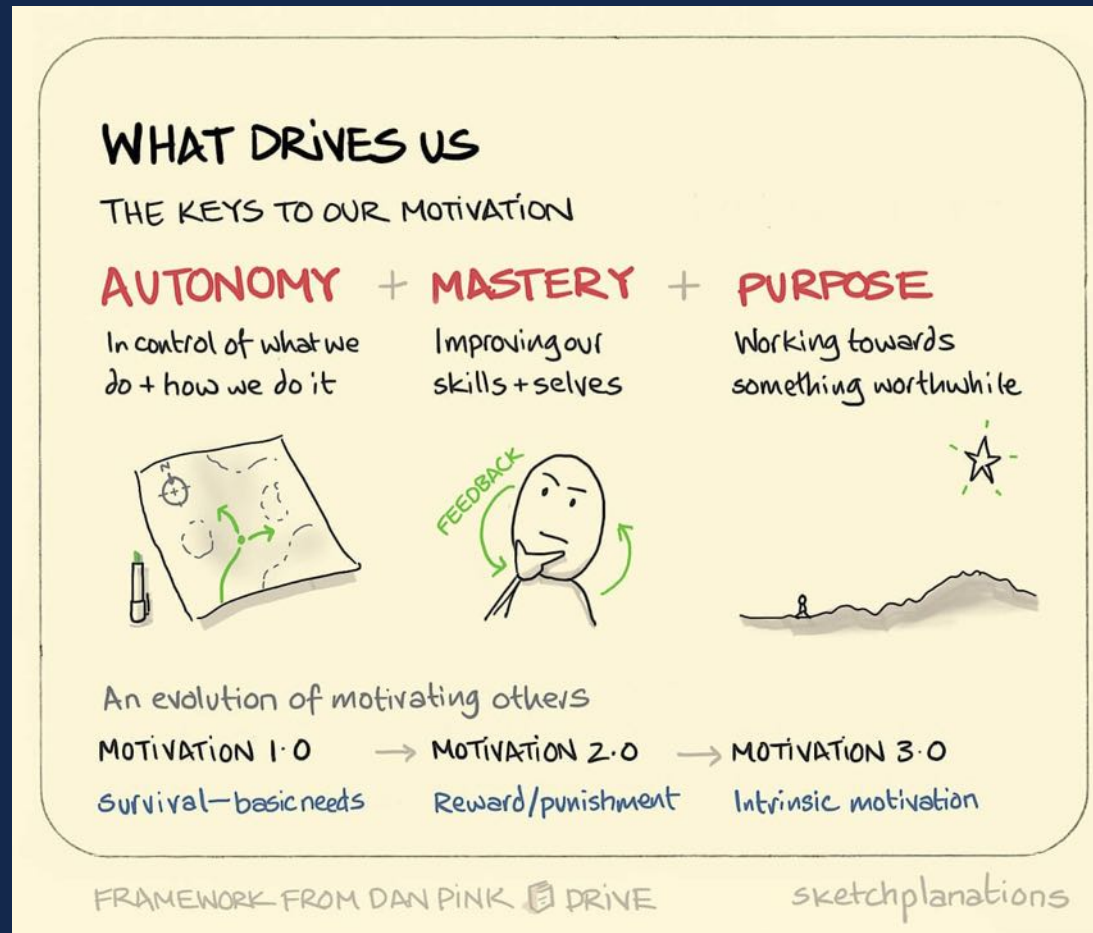
Tips on Living HRT

- Lose the ego
- Giving and receiving criticism feedback
- You aren't your code
- Fail fast and iterate
 - Not just for code but for social interactions
- Think Binary
 - Change the scale and make it a 0 and 1 outcome
 - Something simple, did you do it yes or no not something large that leaves room for failure
 - Doorknob turning ("yeah, it opened!") vs a self-critical discussion of doorknob turning ("my wrist wasn't at a 37 degree angle and it took more than 1second and plus I wasn't really happy about it")

More Tips

- Leave time for learning
- Be patient
- Learn to listen
- Be open to influence
 - Changing your mind isn't weak ,it's mature
 - Say I don't know
- Think Soccer, ~~Dempsey~~ Pulsic may have scored the goal, but the team won not just ~~Dempsey~~ Pulsic.
 - The team plays not just the “star” player (you or otherwise)
 - The strike thru was purposeful what was it about?

Motivation



Seems to only address single person (deck move?)...note group changes

Motivation Applied

	Autonomy	Mastery	Purpose
Development Team	Deciding how to code a functional requirement	Software languages, frameworks, architectures, tools	Delivery of changes, value user feedback
Operations Team	Deciding how to build the infrastructure to meet the technical requirements	Operating Systems, RDMS, routers, firewalls, hardware, tools	Delivering infrastructure with dependable performance and fixing it
DevOps Team	Deciding the best way to meet system functional and non-functional requirements	All of the tools, frameworks, deployment pipeline methods, support, and monitoring technologies.	Delivery of end user happiness including both the system features and the system performance (reliability, scalability, etc.)

Law: Group Size Drives Coordination Time

In a team setting, it's impossible for a group of people to coordinate what needs to get done without spending time on it. The larger the team, the more time is needed.

The Making of a Manager
Julie Zhuo



Meetings

- General Tips
 - Only invite people who **need** to be there
 - Have an agenda and share it before the meeting start
 - Be time aware, start and end the meeting on time, if done early, end early
 - Don't let the meeting wander (too much – allow outlets)
 - Back meetings against interrupts or unproductive time slots
- Meeting Notes
 - All Markdown all the time in your repo
- In-person vs. Online
 - Non-verbal communication
 - Example: Remote only vs Local and then remote
- Maker's Schedule vs Manager's Schedule
 - Managers – interrupt for status
 - Makers - 4-hour blocks or more!
 - Headphones as your office door?

Programmer Communication

- Synchronous vs Asynchronous
 - Online allows for asynchronous, real world not so much, tends to be rude
- Meetings
 - In-Person
 - Online – Slack (yeah it has it), Zoom.us, Google Hangout, Skype, etc.
- Group Chat
- 1-1 Chat
- Email
- Email List and Discussion Thread
 - Noisy minority problems
- Design Doc
 - A living document, not stone tablets
- Issues
- Code Comments
- Reviewing Comments

A fundamental mistake

- Your product / program is ultimately about communication, not just with a machine
 - **Machine** – yes it needs code that is syntactically correct, but in some sense, it doesn't care how well developed your code is. It doesn't care, it's not alive. You and other humans care about runtime efficiency, optimization, maintainability, etc.
 - **Developers** – you and other people who read your code and work with you on your code
 - **Stake Holders** – people who guide the product and thus your code and likely are what funds your code
 - **Collaborators** (vendors, interested third parties, loose stake holders) – people how have some vested interest in your code
 - **End users** -

Software >>> Code

Powell's Power Set Theory for Software

- Software is the power set of many things including:
 - Source Code
 - Test Cases
 - Requirements and Plans
 - Issues
 - Documentation
 - Internal and External
 - Supporting Documents and Communication Assets
- A common failure is not addressing a single source of truth for other aspects of your code. SSoT for all the software or build tech debt!

Single Source(s) of Truth

- The (s) suggest it is misnamed but I'll allow for it
- With code we subscribe strongly to a single source of truth – the **Git Repo!**
- Given the power set you need some single source concept for more than code or is it a tree with a primary entry (ex. Intranet/Slack) and then "single sources" for the domain or tasks at hand
- For me this is still open (testing this quarter yet again) and the tooling out there seems to favor a distributed idea
 - Fail? Multi-tooling leads to multitasking = waste

It's Good to Understand Management Even If Not Wanting to Manage

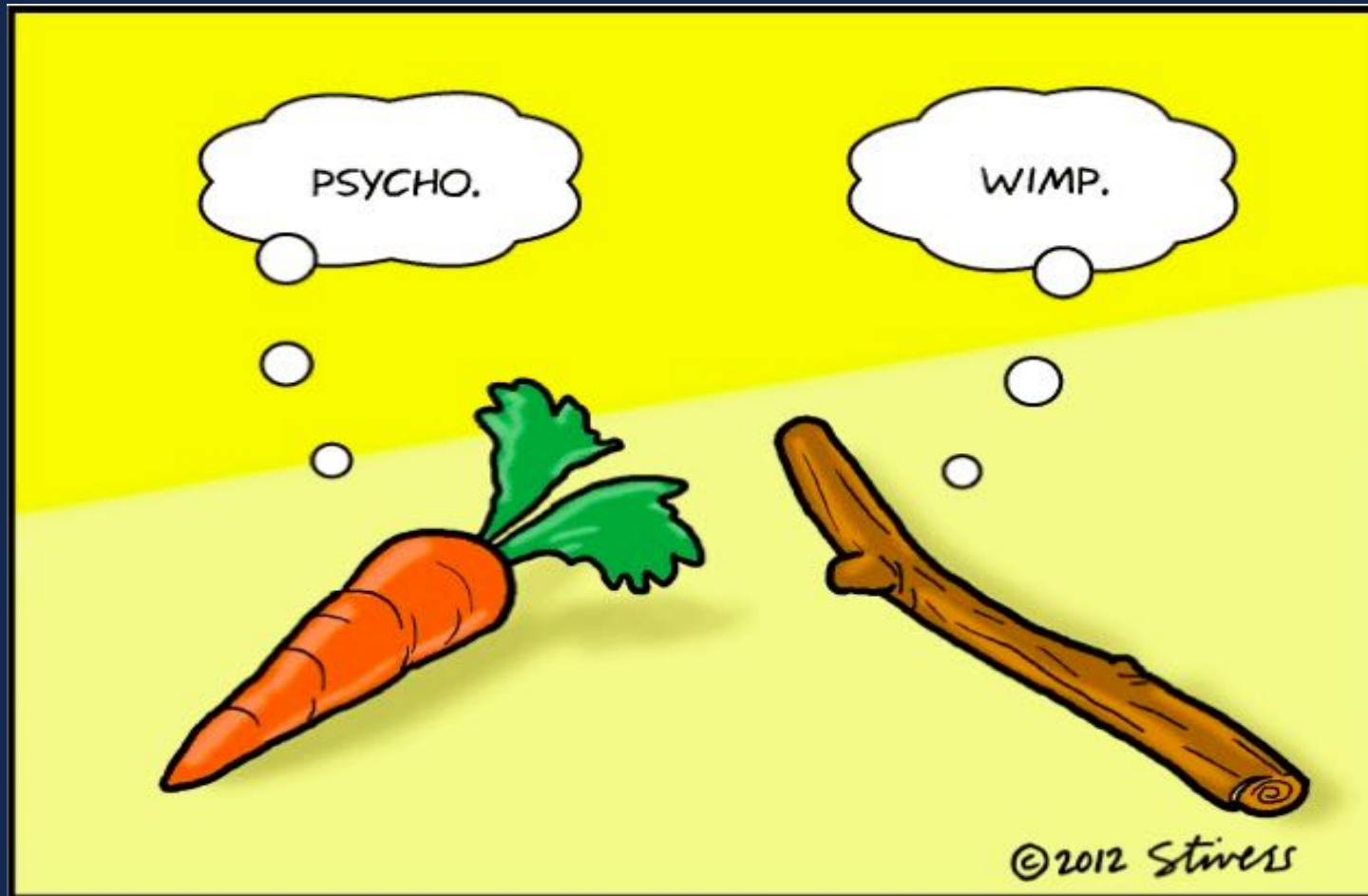
We do need managers

- Like it or not you need managers
- A group of people can do more than a single person in most cases unless extreme disfunction
 - Simple labor output
 - Specialized skill (generalists vs specialists)
 - Full Stack Developers vs Full Stack Aware Developers
- Teams often underperform though because problems of coordination and motivation chip away at collaboration
- Managers (when doing their job) can get better outcomes from a group of people working together

Manager Leader

- In short, we really do need them, without someone in charge we drift around pulled by random currents
 - Reality: A leader will generally happen either by appointment or self-selection
- Even if you never plan on being a manager/leader which is quite fine you would be wise to understand them
 - Helps you understand their decision-making process
 - Leads to better communication and often better reviews (think raises)
- Good idea to understand motivation
 - <https://www.youtube.com/watch?v=u6XAPnuFjJc>
 - This might defy traditional motivation techniques (next slide)

Carrot and Stick



(If You Need to) be a Manager

- Being a leader is tough. It really isn't for everyone, but like all things it will take practice
 - Great opportunity to do so with safety here in school!
- Code is tangible, there is a result and so inherent makers will find it satisfying
- Management lacks quick results and is difficult to measure and can be less obviously satisfying
 - Realization: Less obvious growth except over time, have to know the signs to look for
- Being a manager allows you to scale yourself in some sense, help bring your version out maybe only way to do it
 - You would like humanity to get to Mars? Seems cool, maybe you think Elon might be able to do this, no strike that thought Elon might get his team to help him do it!
 - What kind of manager should we be?

Five Conditions to Foster

Five conditions are likely helpful to improve the team's odds of victory

1. Clear team with boundaries and stable membership
2. Compelling and understood direction
3. Enabling structure
4. Supporting context
5. Expert coaching

Three Areas

- An easy way to think of what is necessary in the team is WWH
 - **W**hy
 - **W**ho
 - **H**ow
- More practically – Apply 3Ps
 - **P**urpose – why are we doing this
 - **P**eople – who are the people that are doing this
 - **P**rocess – how are we going about doing this
- Process should come last.

Respect the Three Time Zones

- We need to live in all three time zones – Past, Present and Future
 - Too much in past leads to ...
 - Too much in future leads to ...
 - Just living in the present leads to ...
- Learning from the past helps keep us from redoing mistakes and guides better decisions
- Planning for the future helps us plot a course and motivate our team in a clear direction
- Living in the present allows us to be grounded to what is important right now

Manager Math: No +, No -, Be *

- Managers should aim not to be just additive
 - The fallacy of the working manager
- Manager should really aim to be multiplicative
 - A good manager magnifies the output of others which is vastly more important than the incremental value they might add themselves additively
- Bad managers might be subtractive
 - Get in the way of the team and reduce the output
- Really bad managers might be divisive
 - Destroy the team with dividing blame

Is Management for you?

- Avoid doing it because it is a “promotion”
- Be a manager because
 - You are at least ok if not like talking to people
 - You are ok trusting and promoting others
 - You think you are stable and calm
- Also
 - You think you have something to share
 - You want to try something different
 - You are up for a challenge

Leadership

- Leadership is a quality not a job
- Managers need to have it, but in reality, all members of a good team should have leadership characteristics
- Leadership is something earned not given
- All students in this class should aim to be a leader in some sense, at the very least in whatever part of the team they end up working on
 - Leader of code, leader of docs, leader of test, etc.
 - Motivation of cross team peering hopefully now understood?

THE 8 TYPES OF MANAGERS

NOW THAT I'VE
MADE NOISE AND
DUMPED ON EVERYONE,
I'M OFF AGAIN



THE SEAGULL

I PREFER
TIMES
NEW
ROMAN



THE MICRO

CAN YOU MAKE
SLIDES FOR MY
AWARD SPEECH
ON YOUR PROJECT?

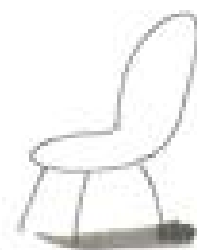


THE SHOW BOAT

USE ONLY
THESE TALKING
POINTS WITH
MY BOSS



THE SPIN DOCTOR



THE ABSENTEE

HOPE YOU
DIDN'T HAVE
WEEKEND
PLANS



THE PITCH FORK

BEFORE WE DECIDE
THE OFFSITE THEME,
LET'S REVIEW THE
GARTNER HYPE CYCLE



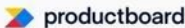
THE PONTIFICATOR

I MADE
CUPCAKES TO
GO THROUGH
YOUR REVIEW

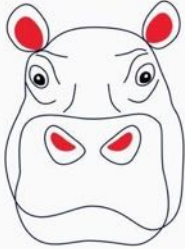


THE PLEASER

Seriously Watch For The Animals!



The dangerous animals of product management



HIPPO
Highest Paid Person's Opinion

It can be tempting to give in to the HIPPOs (founders or CEOs who want to make all the decisions), but don't let them steer you off course. **Bring everything back to your vision and objectives**—if the HIPPOs aren't aligned with these, you could be headed for dangerous waters.



WOLF
Working on Latest Fire

The WOLF has a short attention span and a temptation to jump from one problem to the next. This will **disrupt your team's focus and effectiveness**, making you easy prey for your competitors. Create a **process for collecting feedback** about problems or bugs and only consider these along with all other requests.



RHINO
Really Here in Name Only

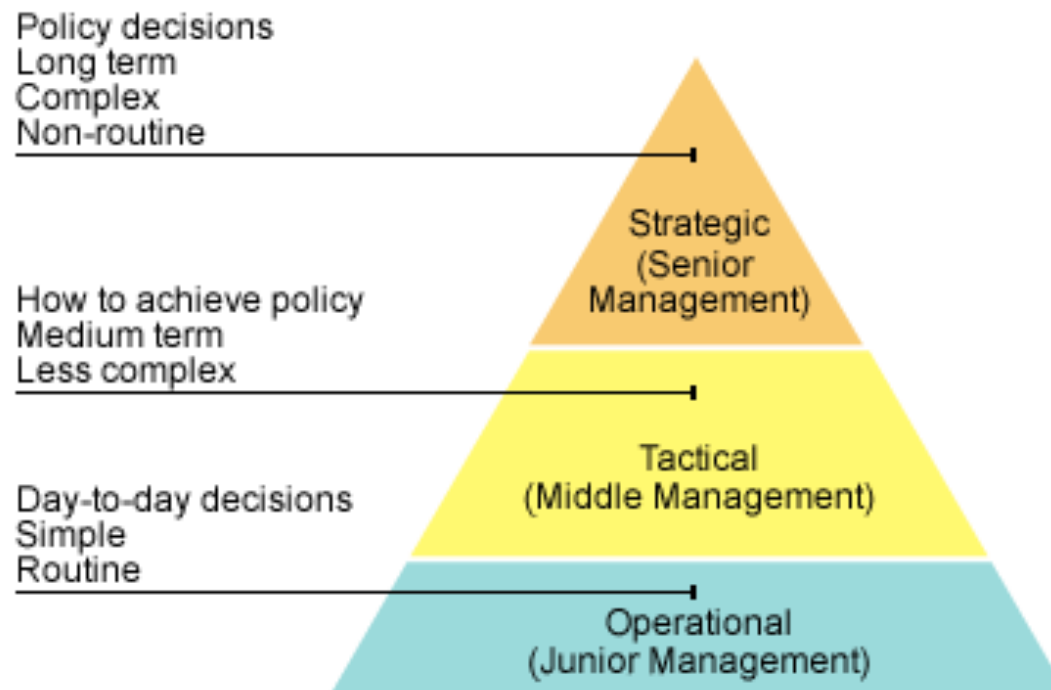
The RHINO is just there to collect a paycheck without contributing much to the team. They might not be actively impeding your decision-making, but they're not helping much, either. Having a **clearly defined prioritization process** can help ensure all your team members understand how decisions are made & give them the confidence to actively participate.



ZEBRA
Zero Evidence But Really Arrogant

ZEBRAS think they know it all but rely on their instinct rather than any actual evidence. To stave off the ZEBRAS in your midst, make sure that you've got data to back up your decisions. Come up with **quick experiments you can run to test ideas and gather evidence**.

Management Pyramid



Communication and Control Patterns

- Chain of Command
 - Hierarchy, command and control
 - Predictable
- Network of Communication
 - Useful for resiliency and error correction
 - Unpredictable
- Use both but understand heavy CoC can destroy or slow down the network paths

Tips on Being a Decent Manager

- **First** - adopt the right attitude to be a great leader
 - servant leadership vs management
- **Second** – don't lose sight of your managed craft
 - Leading from the front so to speak or bags on contractor, coding manager, etc.
- **Third** – lead from the front
 - Act don't just tell, always be there for the team (see #1)
- More tips on trying to be a reasonable leader
 - Lower your ego
 - Be calm - change is constant anyway, don't freak out!
 - Ask lots of questions

More Tips

- Listen
 - My master debugging technique for some team members
- Have the right attitude
 - Can do, as opposed to can't be done
 - Yes, but vs Yes, and...
- Praise in public and criticize in private*
 - * Sometimes public 'floggings' have a use, but generally only to bring group back in line usually after the anti-pattern of not dealing with low performers or letting a toxic person in the group
- Be a catalyst
 - Set up the situation, accelerate a reaction
- Allow failure if it leads to growth (personal or technical)
- Be a teacher and a mentor

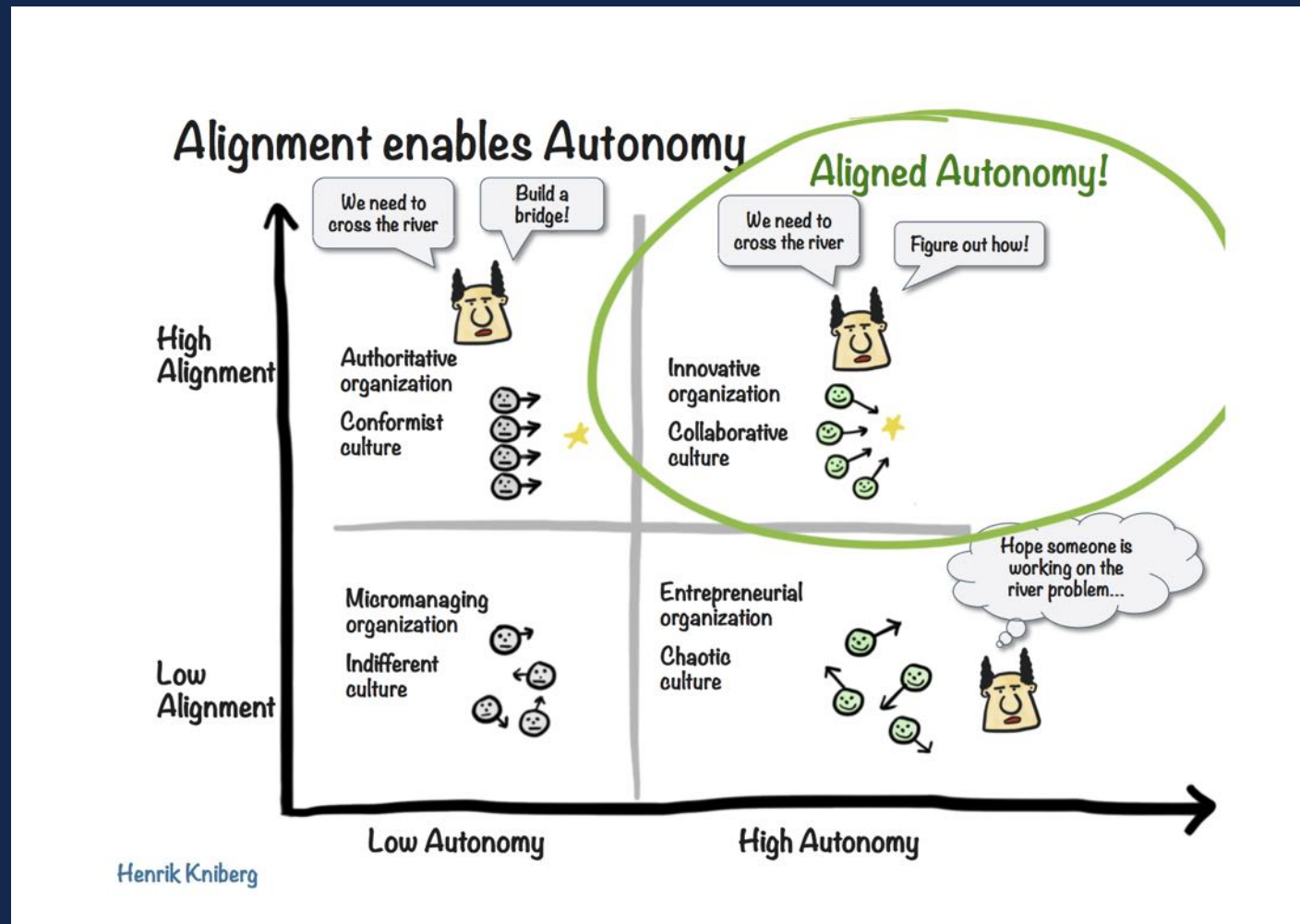
And even more tips

- Set clear goals
- Be honest with people (and yourself)
 - Doesn't mean tell everything though!
- Criticism Sandwiches
 - Positive, Negative, Positive
 - Serve with the right amount of bread!
 - Some people need lots of bread, others much less, but in general too much bread and the meat is likely lost or avoided
- Track Happiness
- Delegate, but get your hands dirty
- Seek to replace yourself
 - ... or at least have a vacation back-up! – BUS FACTOR FOR YOU!
- Know when it is time to make waves
- Fake it until you make it...really
 - You are a phony, we all are, just don't tell anyone 😊

Tips Vary

- “People are like plants” theory
 - Cactus, orchid, etc. they each have different care/feeding instructions
 - I like to think of managers more as coaches, you should try to bring the best out the players you have. Respect each and their differences and adopt your style to what works for them
- The best motivation is intrinsic motivation, though if you have a billion dollars to share with me let me know
 - Supposedly this motivation comes from giving people autonomy, mastery and purpose
 - Autonomy ability to be involved and make choices
 - Mastery ability to grow and master skills
 - Purpose make an impact
 - The purpose one gets really blown out these days with changing the world. That’s great, but self-awareness needs to be practiced. A social sharing app for food pictures isn’t going to change the world it may change the world of how people discuss dinner. Be precise and refine your words. Also revisit the janitor story from earlier

Pro Tip - Get Agreement on Direction



Manager Anti-Patterns

- Don't hire pushovers
 - Be careful with non-pushovers though they can be toxic people in disguise
- Don't ignore low performers
- Don't ignore human issues
- Understand you can't be everyone's friend
- Don't treat your team like children (nor prisoners)

Caesar Your Leader?



Joem hadn't really
@JoemWasTaken



Around 60 people agreed to stab Julius Caesar. At the end of it, Caesar's body only had 23 stab wounds. Even back then, not all the members for a group project would do their part.

Back to The Broader Group Material

Poisonous People Habits – All Members

- Lack of respecting other people's time
 - Not reading the docs before asking when they knew they could, assuming person is available on their odd schedule, etc.
- Ego
 - Overriding consensus decisions repeatedly often using negative logic and corner cases,
- Entitlement
 - Watch out here, age and generational differences abound!
- Immaturity
 - I don't agree with this one as humans like humor (to a point) HOWEVER... there is a point when SirHacksAlot's excessive Giphy posts to the main Slack channel or emoji laden commit messages can get on the team's collective nerves
- Paranoia
 - The “company” or “management” IS out to get you
- Perfectionism
 - See last lecture

IF YOU BITE IT AND YOU DIE; IT'S POISONOUS



VIA 9GAG.COM

IF IT BITES YOU AND YOU DIE; IT'S VENOMOUS

Anti-Venom

- Effective $X\%$ of the time where X is an integer somewhere above 0 and likely well below 100
- Don't feed the trolls
- Check your own emotions
 - Remember self-Postel you can only control yourself
- Find the grain of truth in the venom laced bile
- Kill them with niceness
- Know when to give up
- Play the long game

Venomous Groups

- The culture of a group/organization can be poisoned as well
- We should try to change the culture if we can if not we need to get out lest it gets us
- First problem, ideal situations don't really exist at least for long!
 - Ever heard of Hedonic Adaption?
 - Things often look better from the outside
- Honestly, things tend to vary
 - Toxicity inside a good organization, oasis of sanity in bad ones
 - Toxicity can change over time, what starts out toxic may get ok with familiarity. Good things of course can be perverted as well

Practical Thoughts on Our Groups

- Team Name
- Mascot / Personality
- Team Mission Statement
- Team “By-Laws”
 - No a-holes is not a reasonable by-law folks!
- T-Shirts, stickers and group focused swag might just be digital this quarter
- Getting to know the people beyond the code
- Developing the witty back and forth (...or not)
 - Professional code soldiers aren’t necessarily throwing off one liners like Arnold Schwarzenegger as they slaughter bugs
- War Stories* - come later

Some Debt Foreshadowing

- We will talk a bit in the course about *technical debt*
 - Roughly the interest of technical decisions made that build over time
 - Spoiler: It's bad, but sometimes you want to take it on
- However, as we are talking about groups now we would be remiss to not talk about org debt which is not widely talked about in SWE circles as much as tech debt.

Organizational Debt is Common

- Software Engineering debt doesn't just include the debt of architecture, tech decisions and code shortcuts but should also include organizational debt
- Organizational Debt relates to not working on team issues, processes, procedures, etc.
- Some org debt is due to the emphasis and leadership habits of the organization (sales, tech, etc.)
 - Retros in agile try to help us pay this debt! So do them!

Theory: Most technical debt really is just the result or symptom of an organizational debt

Leadership Fail Patterns By Emphasis

- Organizations run by technical minded individuals may overly value technology and under value the money (ex. Sales) that funds things
- Organizations lead by money minded individuals may under value technology which is what really encourages adoption and money
- Design focused leaders might take too much pride in design at the expense of actual firm or user needs. The “pride of one’s creation” problem.
- Overly analytical leaders may miss subjective measures that overwhelm what data may be “saying” or how users might actually be feeling. Ex: “McDonald’s Fallacy” applied to usage numbers — More Always = Better?
 - **Extreme Danger Alert** - If you optimize to metrics only and hide behind them thinking “objectivity!!!” you may have inadvertent consequences such as a well meaning but toxic environment (ex. MS stack rank, FB Social outcomes via engagement, etc.)

I DON'T APPRECIATE



A **Fish Rots From the Head** Down (A Note to the Sales Leader) ... The statement “a **fish rots from the head** down” means that leadership is the root cause of an organizations failure and demise. This is true whether that organization is a country, a company, or a sales force.

Jun 21, 2012



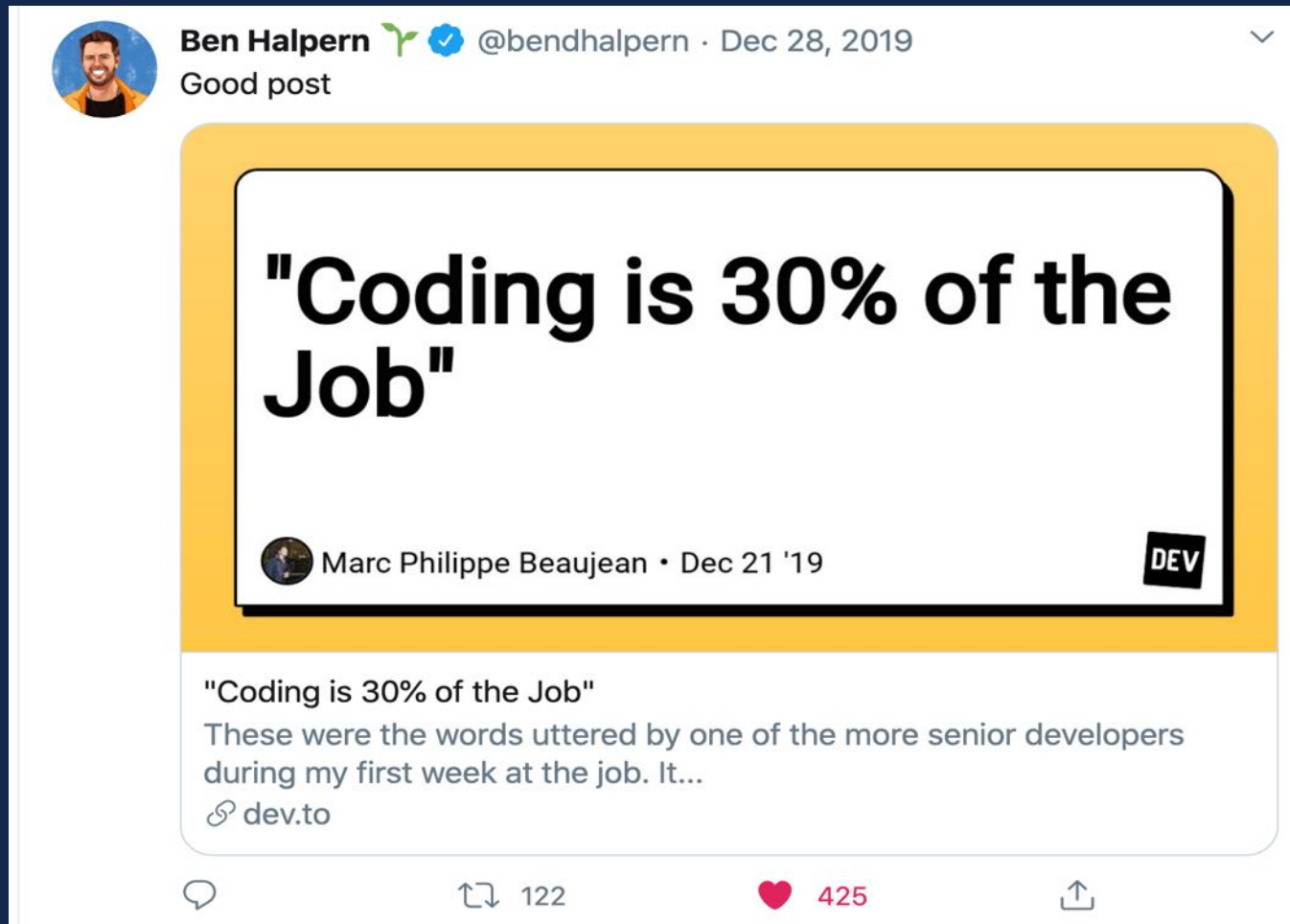
YOUR MEME SIR

Collapse Due to Debt

- In my experience organizational debt is much more dangerous and technical debt is often a result of certain organizational short-comings
 - Undervaluing tech, excessive pivoting, not saying no, etc.
- Paying organizational debt requires creating room for people and habit change
 - Like personal habit change organizational habit change may be more difficult the more ingrained the belief or behavior
- Organizational debt pay down generally needs to be Top-Down and Bottom-Up solely one or the other is likely to fail over time with management imposing beliefs or grassroots become disconnected from organization vision

Wind Down

Well Below 90% Exact Figures Vary



We could do this all day...

```
ws.on("message", m => {
  let a = m.split(" ")
```



Jeff Lembeck @jefflembeck · Aug 26

We, as an industry, really like to pretend that coding is the hard part. We do this when we interview. We do this when we plan and work. It's immensely common and it's so far from the truth.

21

↕ 578

1.9K

[Show this thread](#)

```
ws.send('connect');
}
}else{
  let id = Math.random().toString().slice(2, 8);
  ws.id = id;
  clients.set(id, {client: {positions: {x: 0, y: 0, size: 100, color: 'red'}}});
}
```

Scratch Sometimes



David K.  @DavidKPiano · Dec 12, 2019

IMO, software development is 90% human and 10% code.

Sometimes we spend too much time on the code and not enough time on the human things that really matter.

Automate away the code part as much as you can. Focus on the human part.

 17

 115

 536



[Show this thread](#)

It's Just Skills



Jen Heemstra  @jenheemstra · Jan 5

I've stopped using "soft skills" and started saying "professional skills" instead. The ability to communicate, organize, motivate, etc are essential - they are professional skills because it's hard to be a successful professional without them.



Academic Chatter @AcademicChatter · Jan 5

I dislike the phrase "soft skills", it's often confused with "easy" or "light". Neglecting social/leadership/communication skills and leaving them up to the individual to consider years later can lead to dangerous environments. Recognize their importance and take them seriously.

 69

 617

 2.7K



Finally...Team Maturity will Improve Technical Maturity!



Jason Gorman @jasongorman · Jan 13, 2021

For the best teams, code reviews are about building consensus on standards, not about enforcing standards. Because the best teams don't need enforcement.



Codemanship @codemanship · Jan 13, 2021

Four stages of maturity in team software design:

1. Code reviews needed, but not done
2. Code reviews done infrequently (e.g., once a sprint)
3. Code reviews done frequently (e.g., on every commit)
4. Frequent code reviews not needed

[Show this thread](#)





**KEEP
CALM
AND**

**COLLABORATE FOR
GREAT SOFTWARE**

Coming Up in CSE 110

- And now....
 - What exactly is this project you keep talking about?
- Into Next week
 - Exploratory Programming with an LLM!?!
 - Quality and SE Preview
 - User-Centered Thinking
 - Process models and Design Process
 - TL;DR - Plenty of work AND hopefully fun